JENKINS CI/CD PIPELINES CHALLENGE

1.Create one Declarative pipeline job

Source Code: https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

Step 1: Prerequisites

Before creating the job, make sure you have:

- A running Jenkins server and access to its web interface.
- Your GitHub repository: spring3-mvc-maven-xml-hello-world-1.
- Git installed on the Jenkins machine.
- A compatible JDK and Maven installed on the Jenkins machine, or configured through Jenkins tools.
- Network access from Jenkins to GitHub.

Important: If your Jenkins server is running on an EC2 instance, run the installation and verification commands on that Jenkins instance, not on your Windows computer.

Step 2: Verify Java, Maven, and Git

Log in to your Jenkins EC2 instance using SSH or your terminal and run:

java -version

mvn -version

git --version

Expected output should show installed versions of Java, Maven, and Git.

If Maven or Git is missing, install them according to your EC2 operating system. For Amazon Linux 2023, you can try:

sudo dnf install -y git maven

Check Java again:

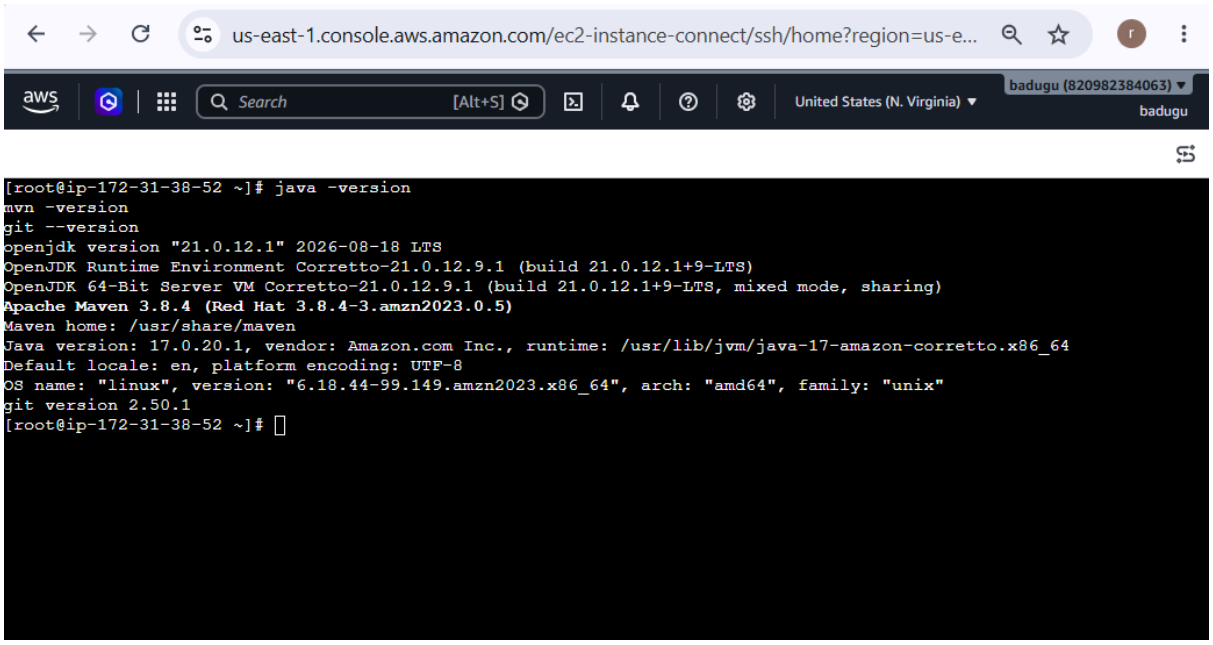
java -version

mvn -version

If Java is missing, install a compatible JDK, for example:

sudo dnf install -y java-21-amazon-corretto-devel

If your project requires an older Java version, use the version supported by its pom.xml.



Step 3: Check the GitHub repository

Open your repository:

Open the GitHub project

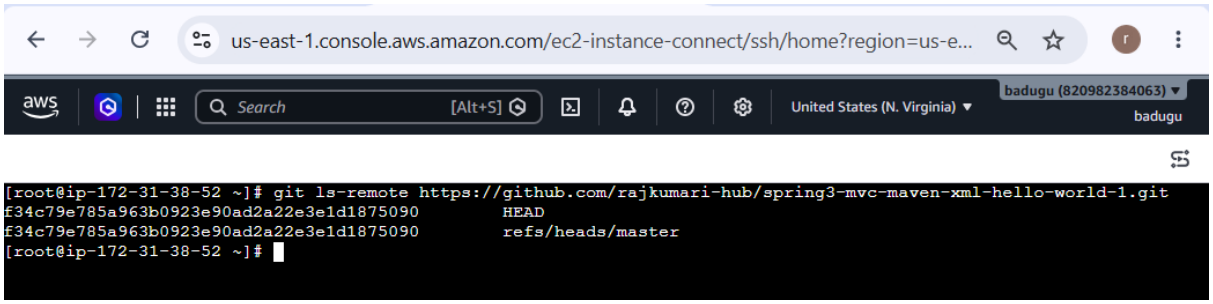
Check whether it contains:

- pom.xml — Maven project configuration.
- src/ — application source code.
- Jenkinsfile — optional pipeline definition stored in the repository.

You can verify the repository is publicly accessible by running:

git ls-remote https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git

If this command displays commit hashes and branch references, Git can access the repository. If it fails, resolve the GitHub URL, network, or repository access issue before continuing.



Step 4: Install required Jenkins plugins

1. Log in to Jenkins.
2. Click Manage Jenkins.
3. Open Plugins or Manage Plugins, depending on your Jenkins version.
4. Select the Available plugins tab.
5. Search for and install these plugins if they are missing:
    - Pipeline
    - Pipeline: Declarative
    - Git
    - Pipeline: Stage View (optional, for visualizing stages)
6. Restart Jenkins if requested.

Jenkins normally includes Pipeline support through its suggested plugins.

Step 5: Create one Declarative Pipeline job

5.1 — Open New Item

Go to the Jenkins Dashboard and click New Item.

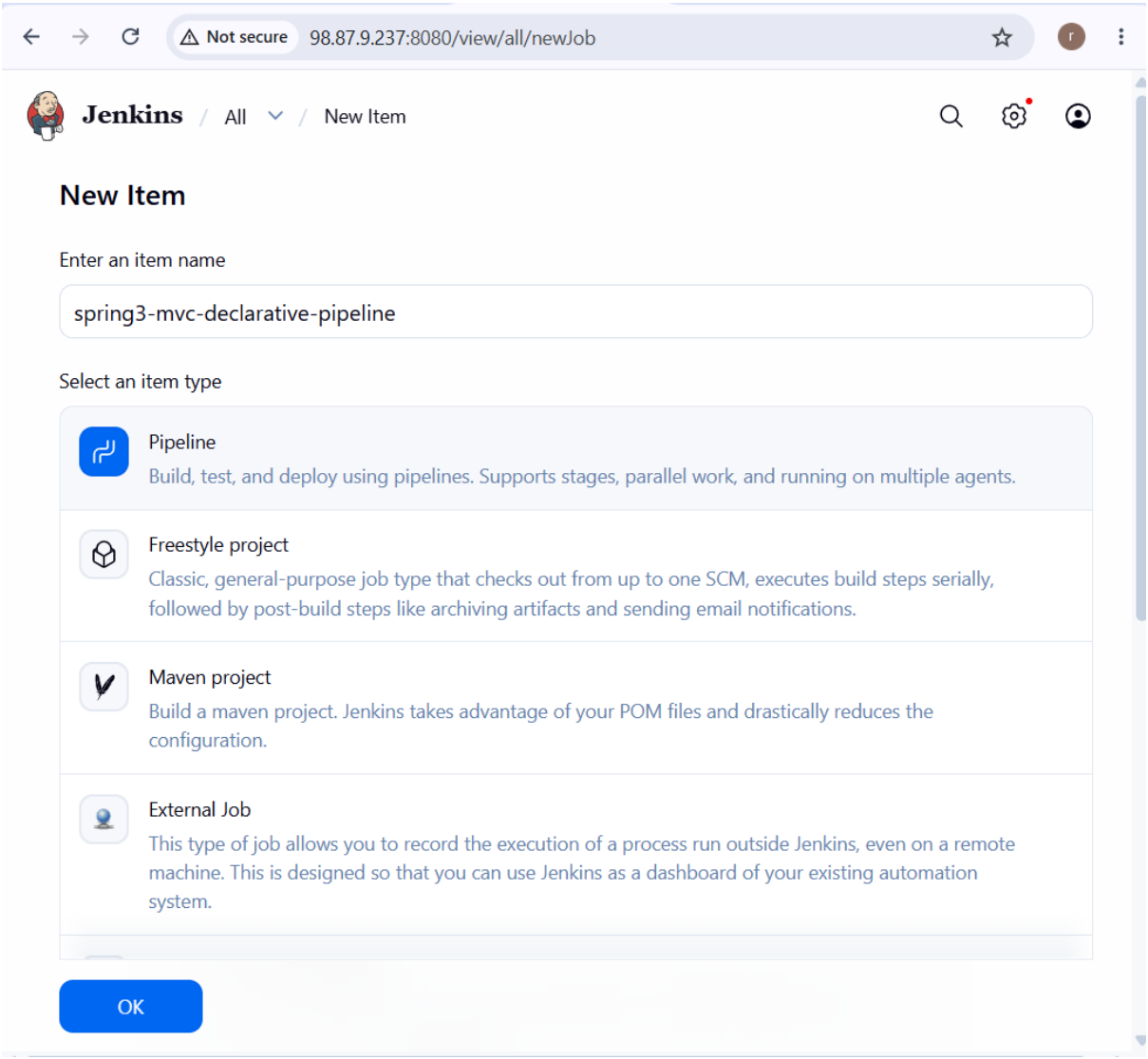
5.2 — Enter the job name

Enter:

spring3-mvc-declarative-pipeline

Select Pipeline, then click OK.

You have now created the job configuration page. We will enter the pipeline code in the next step.



Step 6: Enter the Declarative Pipeline script

On the job configuration page, scroll down to the Pipeline section.

Set the following values:

Field	Value
Definition	Pipeline script
Script	Paste the code below

Use Groovy Sandbox Leave enabled if shown

Copy and paste this complete pipeline script into the Script box.

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Downloading source code from GitHub'

                git branch: 'main',
                    url: 'https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git'
            }
        }

        stage('Verify Tools') {
            steps {
                sh 'java -version'
                sh 'mvn -version'
            }
        }

        stage('Build and Test') {
            steps {
```

```
            sh 'mvn -B clean package'
        }
    }
}

post {
    always {
        junit testResults: 'target/surefire-reports/*.xml',
              allowEmptyResults: true
    }

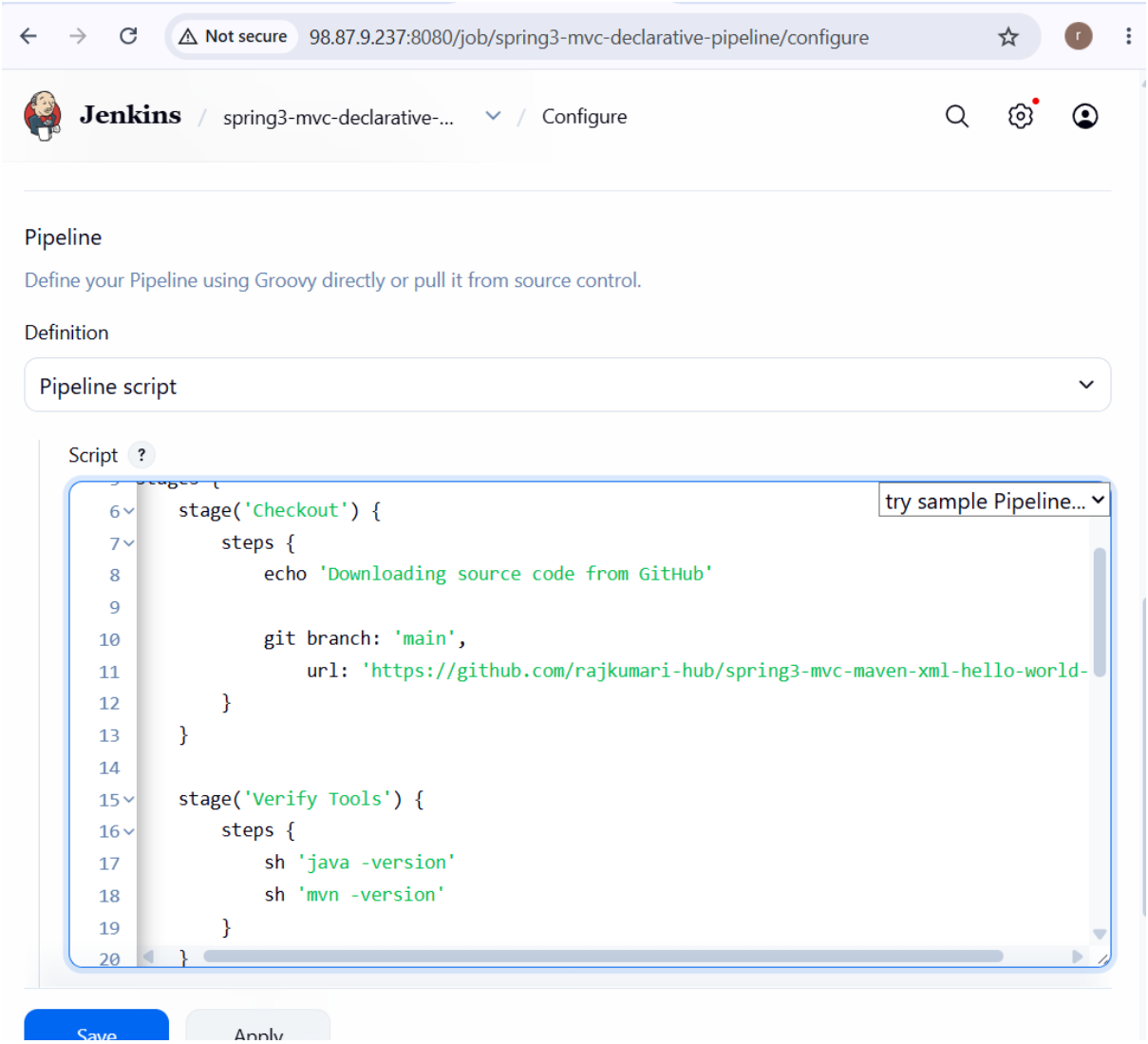
    success {
        archiveArtifacts artifacts: 'target/*.war,target/*.jar',
                         allowEmptyArchive: true

        echo 'Build completed successfully!'
    }

    failure {
        echo 'Build failed. Check the Console Output.'
    }
}
}
```

Important: This example assumes the repository's branch is main. If GitHub shows master or another default branch, replace main with the actual branch name.

This pipeline performs the checkout, verifies the installed tools, builds and tests the application, publishes available JUnit test reports, and archives any WAR or JAR files. It follows Jenkins Declarative Pipeline syntax



Step 8: Run the pipeline

8.1 — Click Build Now

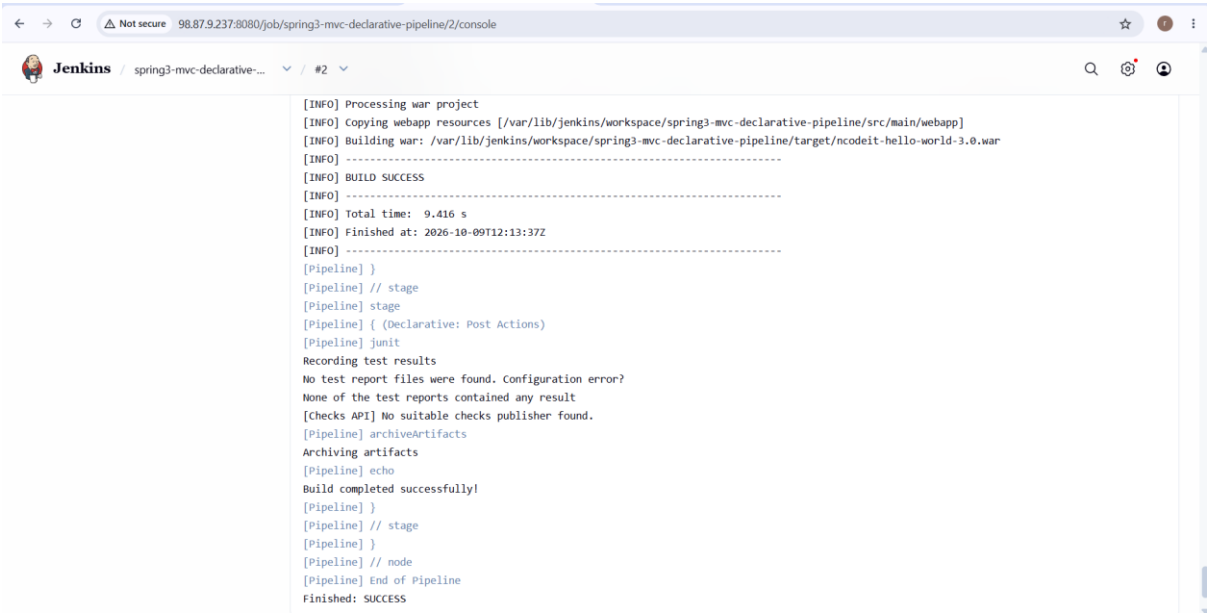
On the left-hand menu of your job page, click Build Now.

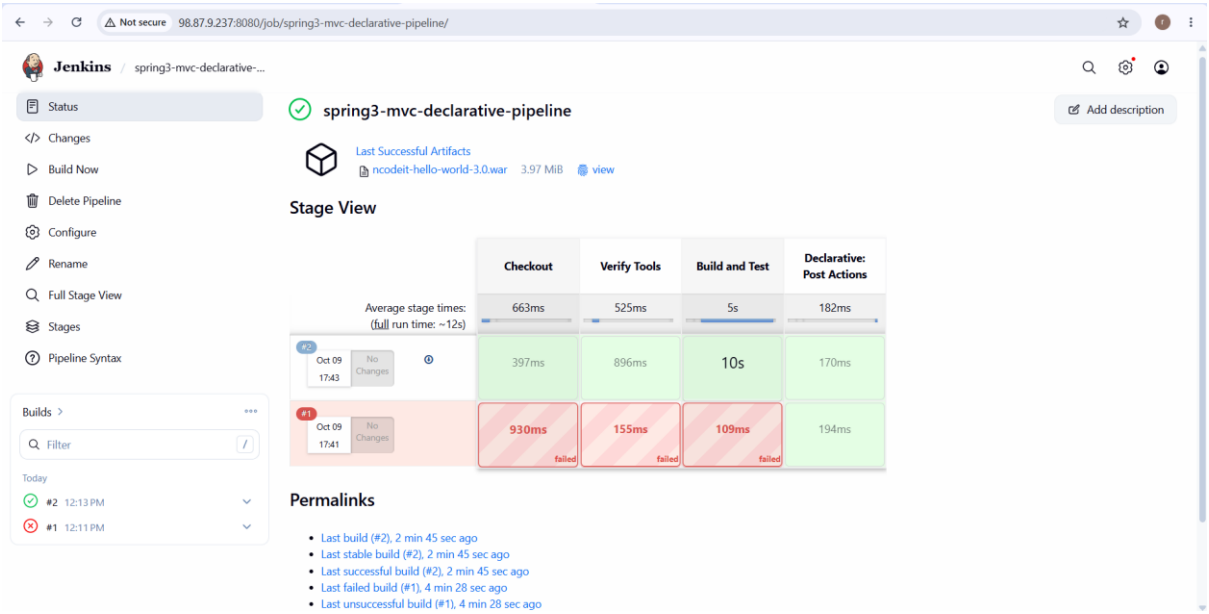
8.2 — Open the build

Under Build History, click the latest build number, such as #1, and then select Console Output.

Step 9: Verify the results

Check that the Console Output contains messages similar to these:





These are illustrative messages; your actual output will depend on your Java version, Maven version, and project configuration.

You should also see the stages Checkout, Verify Tools, and Build and Test in the pipeline stage view.

Step 10: Troubleshooting common errors

Error	What to do
Couldn't find any revision to build	Check the repository URL and branch name.
mvn: command not found	Install Maven or configure it in Jenkins tools and ensure the Jenkins service can access it.
java: command not found	Install a compatible JDK and configure the Jenkins environment.
BUILD FAILURE	Open Console Output and inspect the first Maven compilation, dependency, or test error.
Repository not found or authentication failure	Verify that the repository exists and is accessible. Configure Jenkins credentials if it is private.
No WAR or JAR archived	Check the Maven packaging configuration and the contents of the target/ directory.

Final result

You have one Jenkins Declarative Pipeline job that performs a basic CI build from your GitHub source code.

For reference, Jenkins also supports keeping the pipeline in a repository Jenkinsfile and selecting Pipeline script from SCM instead of embedding the code in Jenkins. That approach is generally preferable when you want the pipeline definition to be version-controlled.

1.Create one Scripted pipeline job

Source Code: https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git

Step 1: Prerequisites

Make sure your Jenkins server has Java, Maven, and Git installed.

Run these commands on your Jenkins EC2 instance:

java -version

mvn -version

git --version

If Git or Maven is missing on Amazon Linux 2023:

sudo dnf install -y git maven

If Java is missing, install a compatible JDK:

sudo dnf install -y java-21-amazon-corretto-devel

Verify the repository is accessible:

git ls-remote https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1

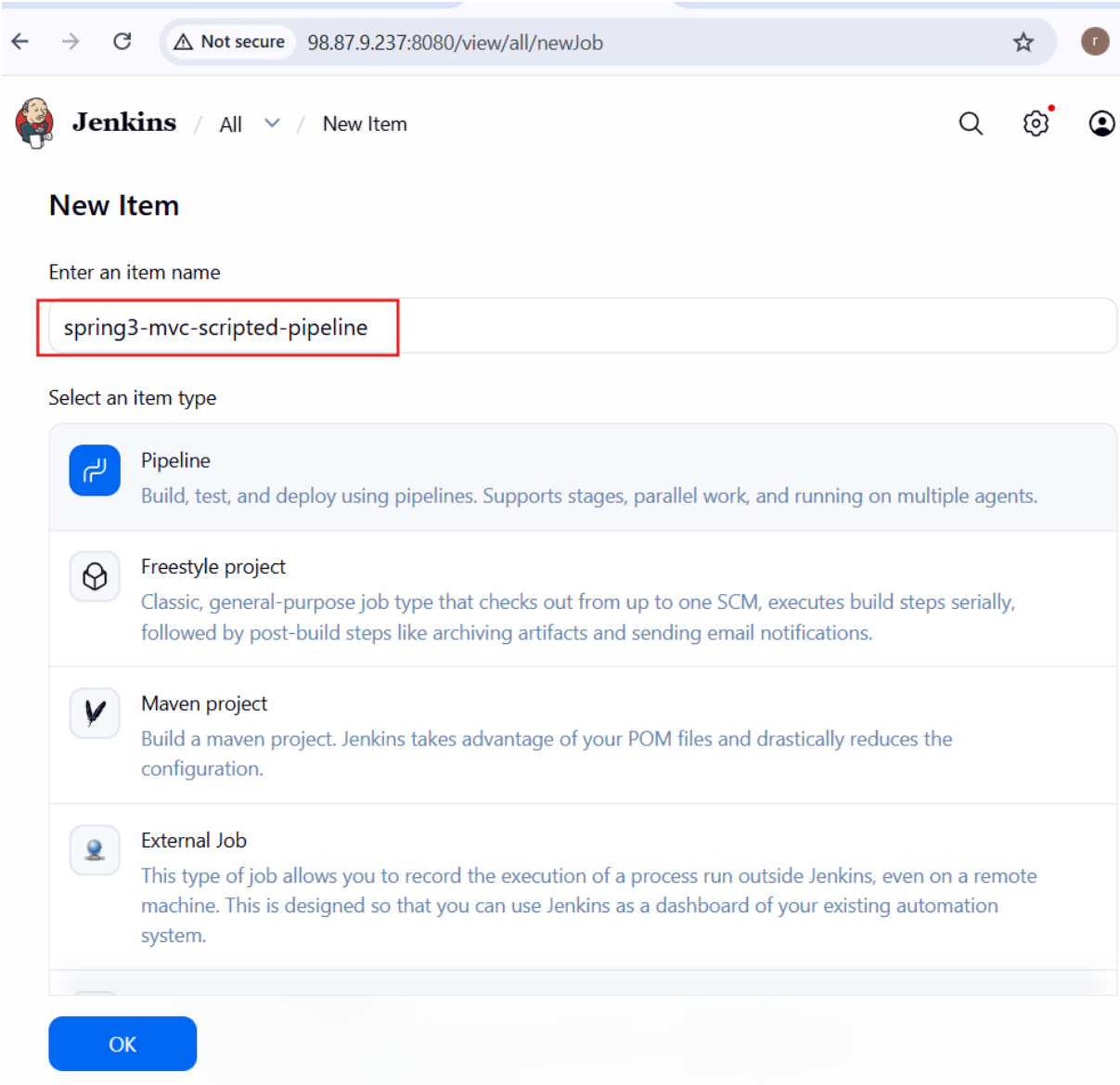
Step 2: Create a new Jenkins job

1. Log in to your Jenkins dashboard.
2. Click New Item.
3. Enter the item name:

spring3-mvc-scripted-pipeline

4. Select Pipeline.
5. Click OK.

Important: Select Pipeline as the job type. Scripted Pipeline is configured inside the Pipeline script field; it is not a separate job type.



Step 3: Configure the pipeline

1. Scroll down to the Pipeline section.
2. Set Definition to Pipeline script.
3. Paste the following Scripted Pipeline code into the Script box.

```
node {
    stage('Checkout') {
        echo 'Checking out source code from GitHub'
```

```
        git branch: 'main',
            url: 'https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git'
    }

    stage('Verify Tools') {
        echo 'Checking Java and Maven versions'

        sh 'java -version'
        sh 'mvn -version'
    }

    stage('Build and Test') {
        echo 'Building the application using Maven'

        sh 'mvn -B clean package'
    }

    stage('Archive Artifacts') {
        archiveArtifacts(
            artifacts: 'target/*.war,target/*.jar',
            allowEmptyArchive: true
        )
    }
```

```
    currentBuild.result = 'SUCCESS'

    echo 'Pipeline completed successfully!'
}
```

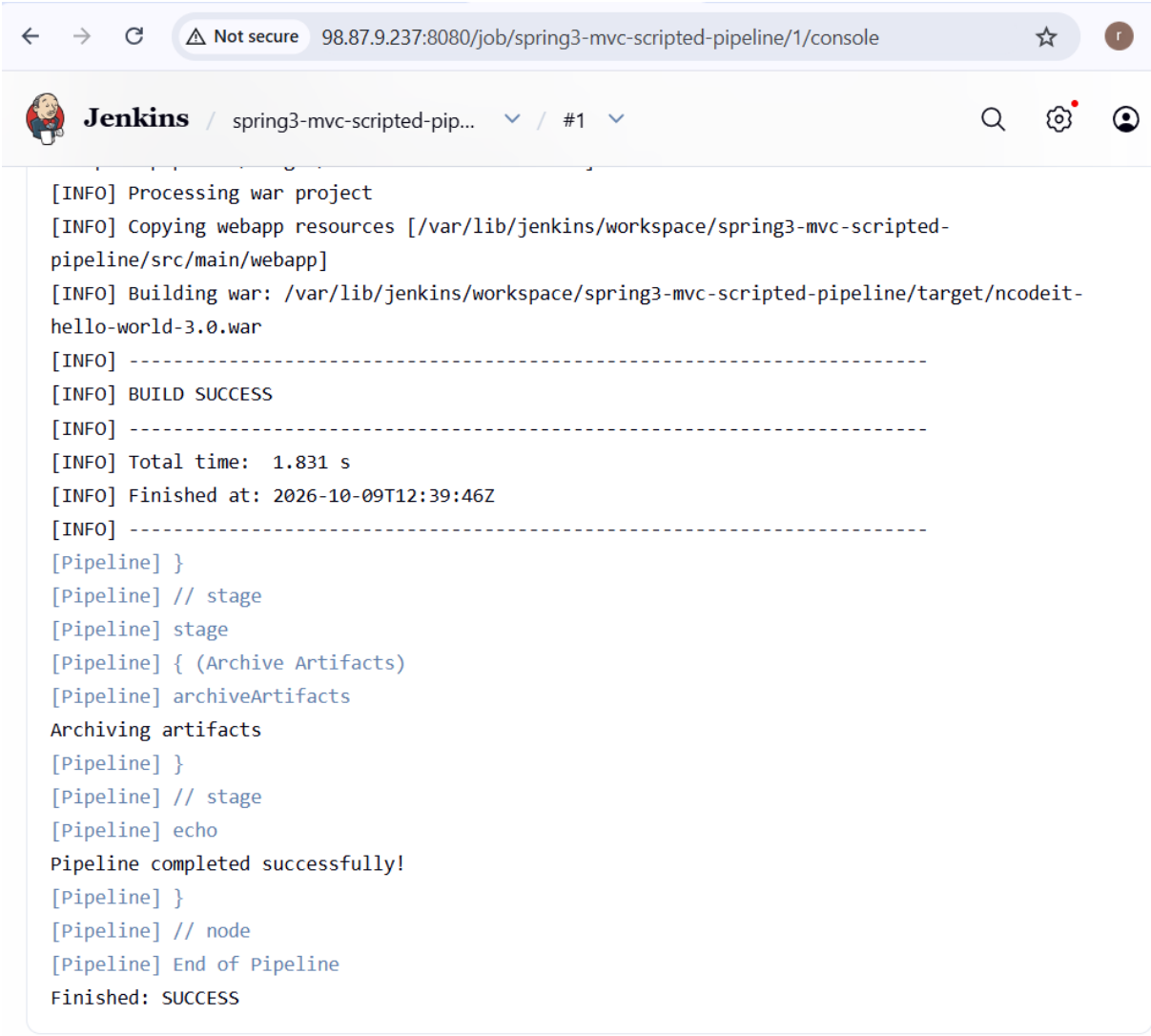
Note: This script assumes your repository's branch is main. Change it if the actual branch is different. This is Scripted Pipeline syntax because it uses node {}, stage {}, and ordinary Groovy control structures rather than the Declarative pipeline {} block.

Step 4: Save the job

1. Scroll to the bottom.
2. Click Save.
3. Jenkins opens your new job dashboard.

Step 5: Execute the pipeline

1. Click Build Now.
2. Wait for the build to start.
3. Under Build History, click the



These are illustrative messages. Your actual output depends on the repository and build environment.

Step 7: Troubleshooting

Error	Solution
Couldn't find any revision to build	Verify the branch name and GitHub URL.
mvn: command not found	Install Maven or configure Maven in Jenkins.
java: command not found	Install a compatible JDK and ensure Jenkins can access it.

BUILD FAILURE	Check the first Maven error in Console Output.
No artifacts found	Check whether Maven generates a WAR/JAR in the target/ directory.

3.Create one Parameterized job

Source code: https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git

Step 1: Prerequisites

Ensure Jenkins has Git, Java, and Maven available.

Run these commands on your Jenkins EC2 instance:

git --version

java -version

mvn -version

If your repository is public, you do not need GitHub credentials for checkout.

Step 2: Create a new Jenkins job

1. Log in to the Jenkins dashboard.
2. Click New Item.
3. Enter the job name:

spring3-mvc-parameterized-job

4. Select Pipeline.
5. Click OK.

Step 3: Enable parameterization

On the job configuration page:

1. Find the General section.
2. Select This project is parameterized.
3. Add the following parameters.

Parameter 1: ENVIRONMENT

Choose the target environment for this build.

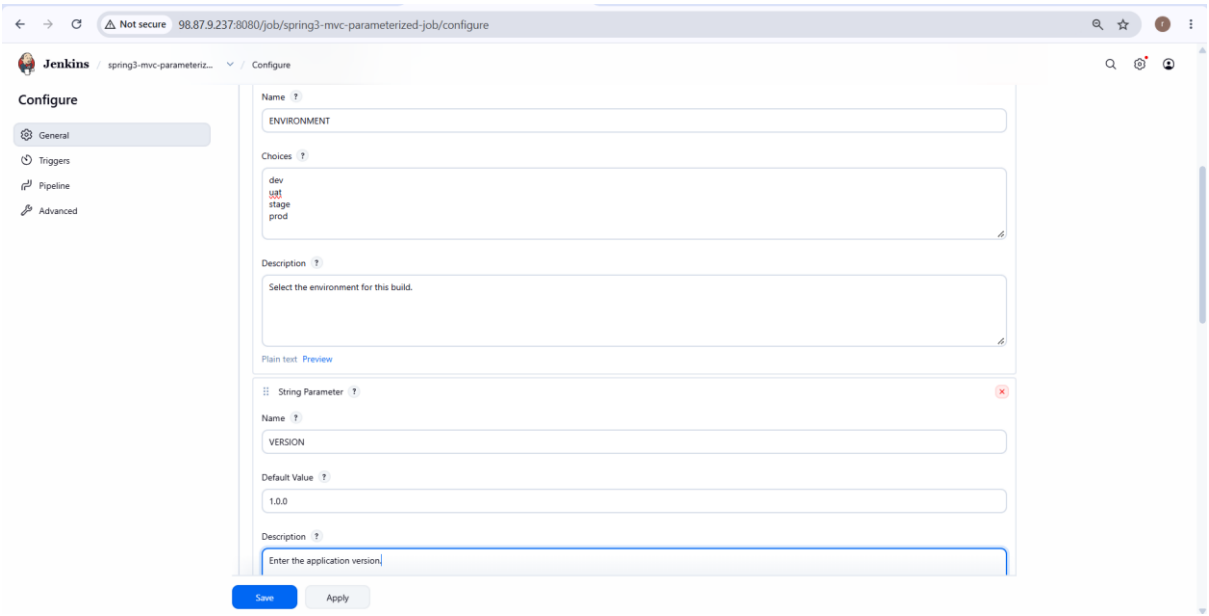
Parameter type	Choice Parameter
Name	ENVIRONMENT
Choices	dev, uat, stage, prod
Description	Select the environment for this build.

Parameter 2: VERSION

Enter the version label for the build.

Parameter type	String Parameter
Name	VERSION
Default value	1.0.0
Description	Enter the application version.

To add each parameter, click Add Parameter, choose its type, and fill in the fields.



Step 4: Configure the Pipeline script

Scroll to the Pipeline section.

- Definition: Pipeline script

- Paste this code into the Script box.

```
pipeline {
    agent any

    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['dev', 'uat', 'stage', 'prod'],
            description: 'Select the environment'
        )

        string(
            name: 'VERSION',
            defaultValue: '1.0.0',
            description: 'Enter the application version'
        )
    }

    stages {
        stage('Display Parameters') {
            steps {
                echo "Environment: ${params.ENVIRONMENT}"
                echo "Version: ${params.VERSION}"
            }
        }
```

```
    stage('Checkout') {
        steps {
            git branch: 'main',
                url: 'https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git'
        }
    }

    stage('Verify Tools') {
        steps {
            sh 'java -version'
            sh 'mvn -version'
        }
    }

    stage('Build and Test') {
        steps {
            sh 'mvn -B clean package'
        }
    }

    stage('Build Summary') {
        steps {
            echo "Build completed for ${params.ENVIRONMENT}"
            echo "Application version: ${params.VERSION}"
```

```
        }
    }
}

post {
    success {
        archiveArtifacts(
            artifacts: 'target/*.war,target/*.jar',
            allowEmptyArchive: true
        )
        echo 'Parameterized build completed successfully!'
    }

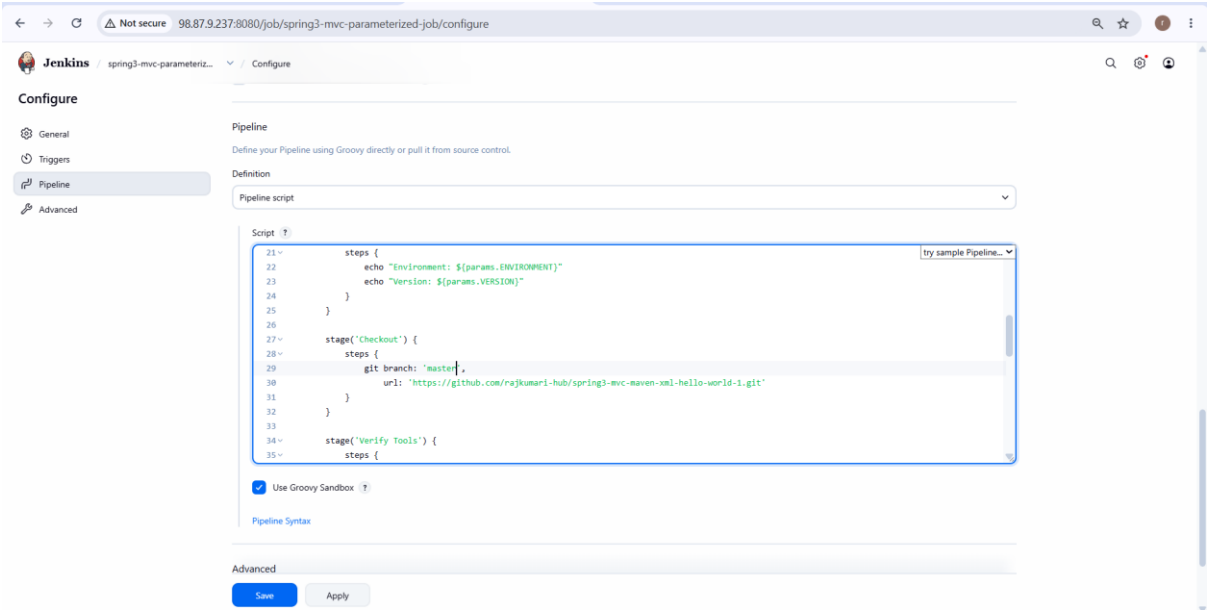
    failure {
        echo 'Build failed. Check Console Output.'
    }
}
}
```

Important: This example assumes the GitHub branch is main. Change the branch if your repository uses a different name.

The parameters block defines the inputs, and params.ENVIRONMENT and params.VERSION read the values selected by the user. The example builds the same application for every environment; it does not deploy to those environments.

Step 5: Save the job

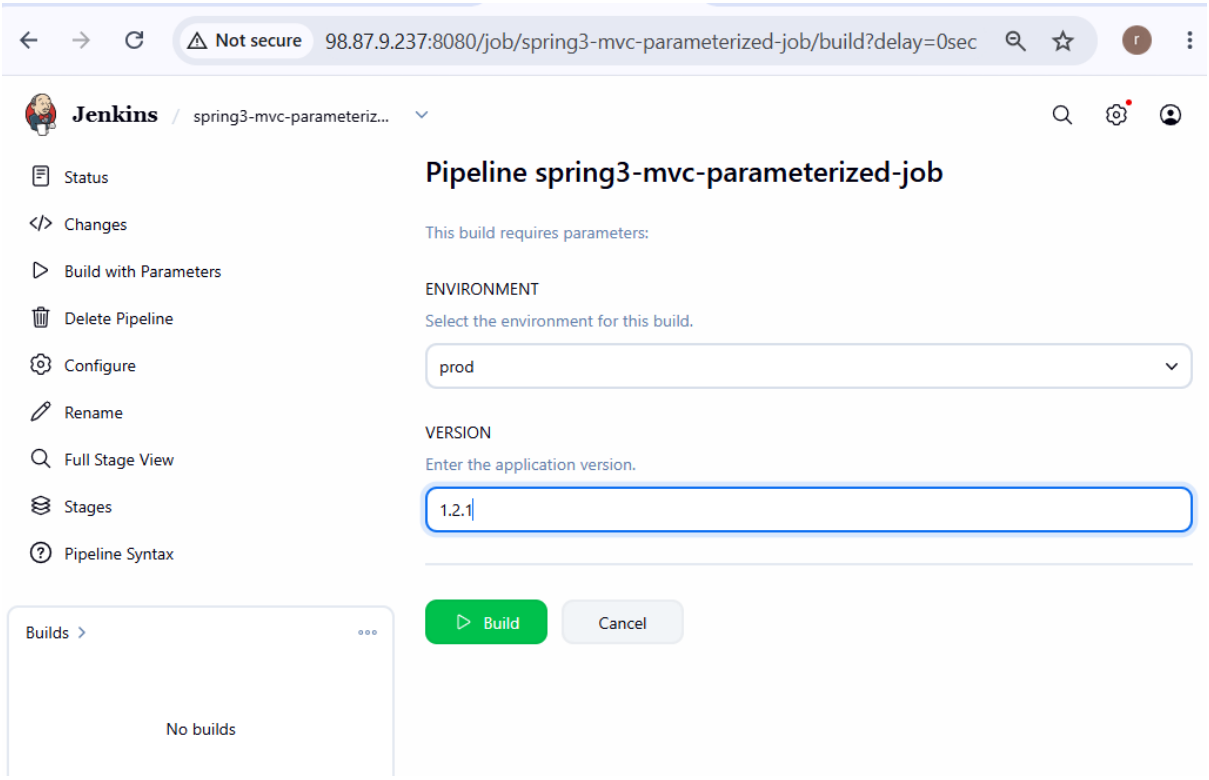
1. Click Save at the bottom of the page.
2. Open the job dashboard.



Step 6: Run the parameterized job

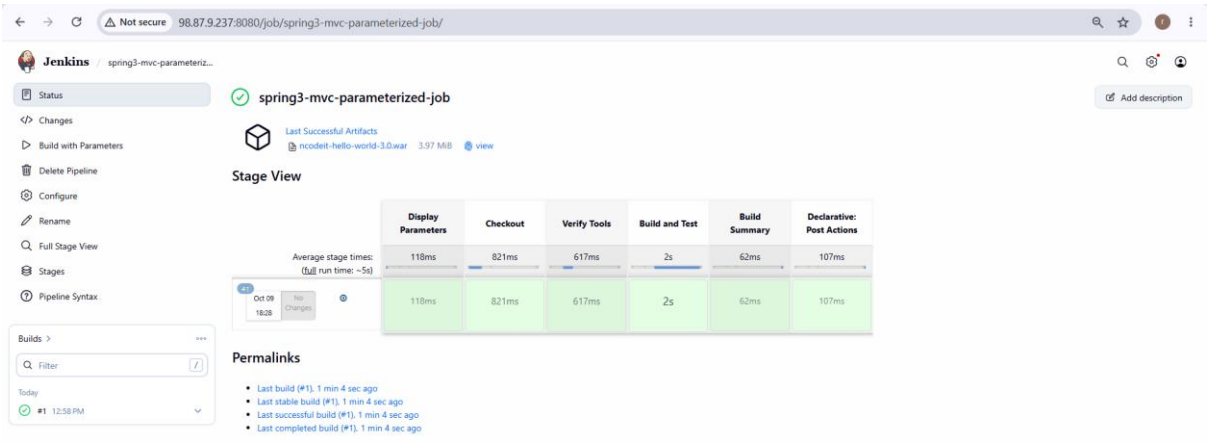
Click Build with Parameters instead of Build Now.

You should see a form similar to this:



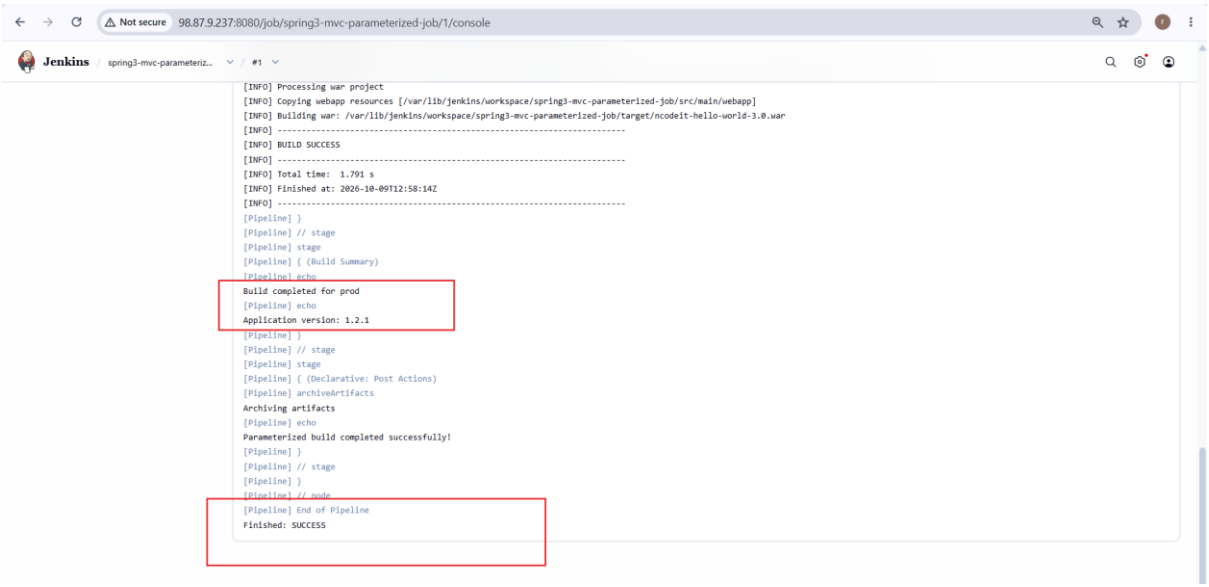
For your first test, select dev and enter version 1.0.0.

Click Build on the actual Jenkins page.



Step 7: Verify Console Output

Open Build History → build number → Console Output.



Step 8: Test with different parameters

Run the job again using different inputs.

Build	ENVIRONMENT	VERSION
#1	dev	1.0.0
#2	uat	1.0.1
#3	stage	1.1.0
#4	prod	1.1.0

Each run records the values selected for that build.

4.Write sample skeleton for Declarative and Scripted pipelines

Declarative Pipeline — Sample Skeleton

Declarative Pipeline uses a predefined structure beginning with pipeline {}. It is generally recommended for beginners because it is easier to read and maintain.

```
pipeline {
    agent any

    environment {
        APP_NAME = 'myapp'
    }

    parameters {
        string(name: 'VERSION', defaultValue: '1.0.0',
               description: 'Application version')
    }

    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out source code'
            }
        }

        stage('Build') {
            steps {
                echo "Building ${APP_NAME} version ${params.VERSION}"
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully'
        }

        failure {
            echo 'Pipeline failed'
        }

        always {
            echo 'Pipeline execution finished'
        }
    }
}
```

Explanation

Keyword	Purpose
pipeline	Defines the Declarative Pipeline.
agent any	Allows Jenkins to select any available agent.
environment	Defines environment variables.
parameters	Accepts user inputs when starting a build.
stages	Groups the pipeline into stages.
stage	Defines a stage such as Build or Test.
steps	Contains the commands or actions to execute.
post	Defines actions to run after the pipeline or based on its result.

Execution flow: Checkout → Build → Test → Deploy → Post actions.

2.Scripted Pipeline — Sample Skeleton

Scripted Pipeline uses Groovy code inside node {}. It gives you greater programming flexibility for conditions, loops, and custom logic.

```
node {
    try {
        stage('Checkout') {
            echo 'Checking out source code'
        }

        stage('Build') {
            def appName = 'myapp'
            def version = '1.0.0'

            echo "Building ${appName} version ${version}"
        }

        stage('Test') {
            echo 'Running tests'
        }

        stage('Deploy') {
            echo 'Deploying application'
        }

    } catch (Exception err) {
        echo "Pipeline failed: ${err.getMessage()}"
        throw err

    } finally {
        echo 'Pipeline execution finished'
    }
}
```

Explanation

Keyword Purpose

node	Allocates a Jenkins executor and workspace.
stage	Organizes the execution into named stages.
try	Executes code while allowing exceptions to be handled.
catch	Handles an exception when one occurs.

finally	Executes cleanup or final actions regardless of success or failure.
def	Declares a local Groovy variable.
echo	Prints a message in the Jenkins Console Output.

Execution flow: Checkout → Build → Test → Deploy → Exception handling and final actions.

3. Key differences

Feature	**Declarative**	**Scripted**
Basic structure	pipeline {}	node {}
Syntax	Structured and predefined	Groovy-based and flexible
Learning curve	Easier for beginners	Requires more Groovy knowledge
Conditions and loops	Supported within the Declarative structure	Flexible Groovy control flow
Error handling	Commonly uses post conditions	Can use try, catch, and finally
Best suited for	Standard CI/CD workflows	Complex, programmatic workflows

5.Create one Declarative job

source code:
https://github.com/rajkumarihub/sabear_simplecutomerapp/tree/feature-1.1

Step 1: Prerequisites

Make sure Jenkins is running and you can access its dashboard. Your Jenkins build agent should have Git and the build tools required by your application.

If you are using your Amazon Linux EC2 Jenkins server, run:

git --version

java -version

mvn -version

These commands check Git, Java, and Maven. If the application uses a different build system, such as Gradle or Node.js, use the appropriate tools instead.

Step 2: Verify the GitHub branch

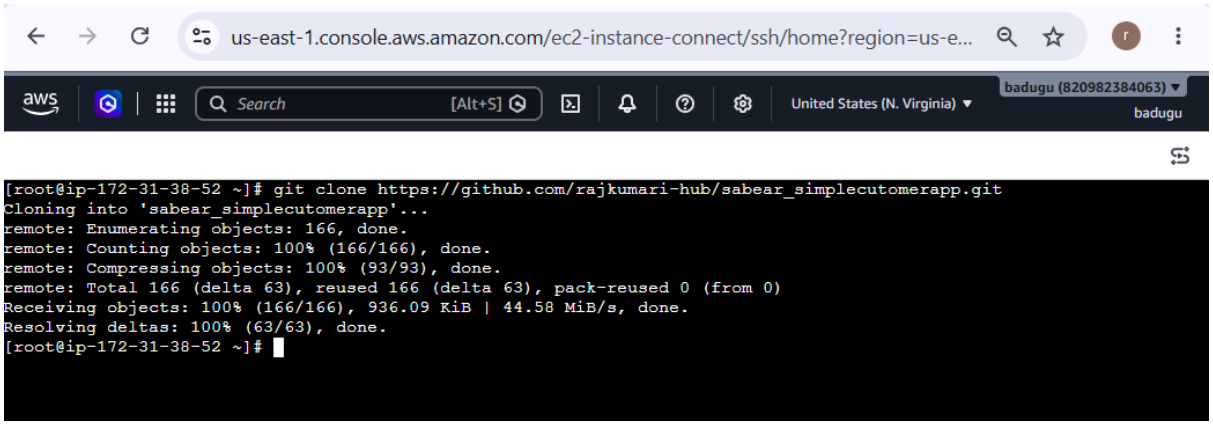
Open your repository and confirm that the feature-1.1 branch exists:

Open the feature-1.1 branch on GitHub

The Git clone URL to use in Jenkins is:

https://github.com/rajkumari-hub/sabear_simplecutomerapp.git

The branch name is feature-1.1. The /tree/feature-1.1 portion belongs to the GitHub browser URL, not the clone URL.



Step 3: Create the Jenkins job

1. Open your Jenkins dashboard.
2. Click New Item.
3. Enter the job name:

sabear-simplecustomerapp-declarative

4. Select Pipeline.
5. Click OK.

Step 4: Configure the Declarative Pipeline

Scroll down to the Pipeline section and configure:

Field	Value
Definition	Pipeline script
Script	Paste the code below

Use this Declarative Pipeline if your repository contains a Maven pom.xml file at its root.

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out feature-1.1 branch'

                git branch: 'feature-1.1',
                    url: 'https://github.com/rajkumari-hub/sabear_simplecutomerapp.git'
            }
        }

        stage('Verify Project') {
            steps {
```

```
            sh 'pwd'
            sh 'ls -la'
            sh 'java -version'
            sh 'mvn -version'
        }
    }

    stage('Build') {
        steps {
            sh 'mvn -B clean package'
        }
    }

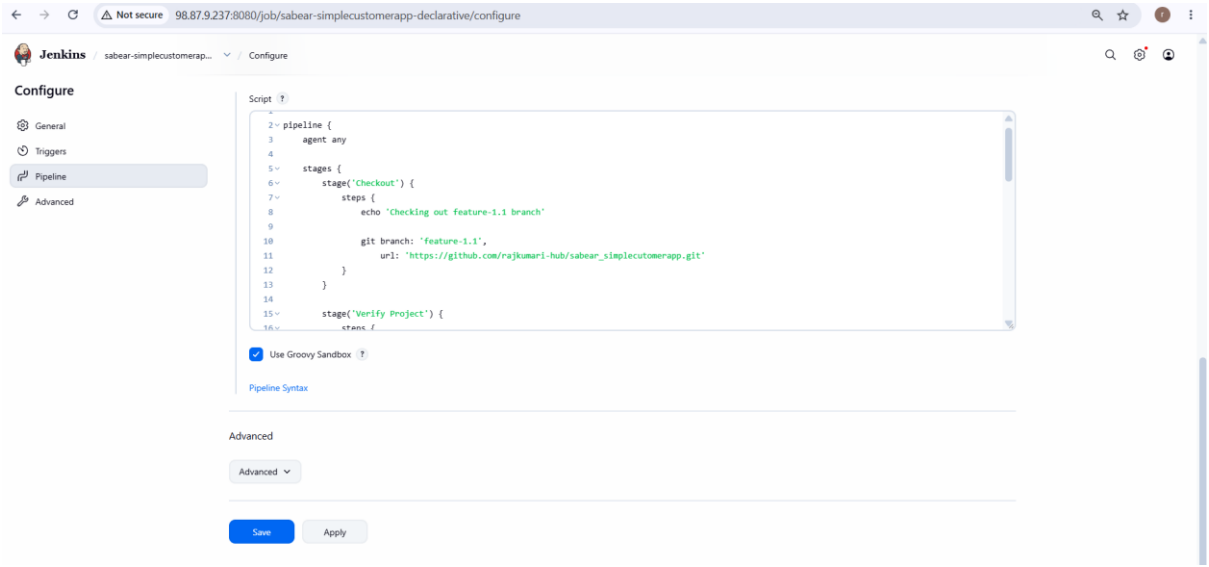
    stage('Archive Artifacts') {
        steps {
            archiveArtifacts(
                artifacts: 'target/*.war,target/*.jar',
                allowEmptyArchive: true
            )
        }
    }
}

post {
    success {
        echo 'Declarative Pipeline completed successfully!'
```

```
        }

        failure {
            echo 'Pipeline failed. Check Console Output.'
        }

        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

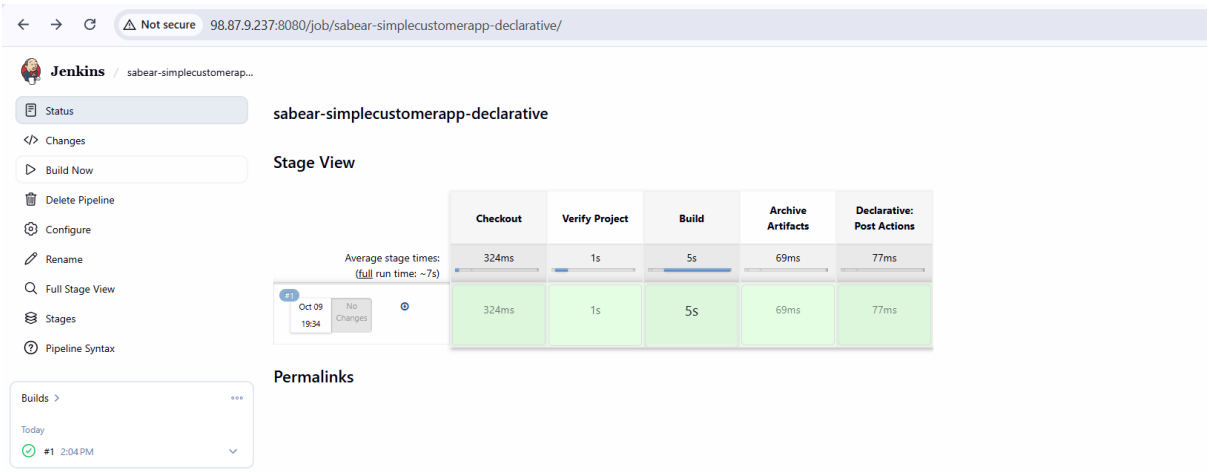


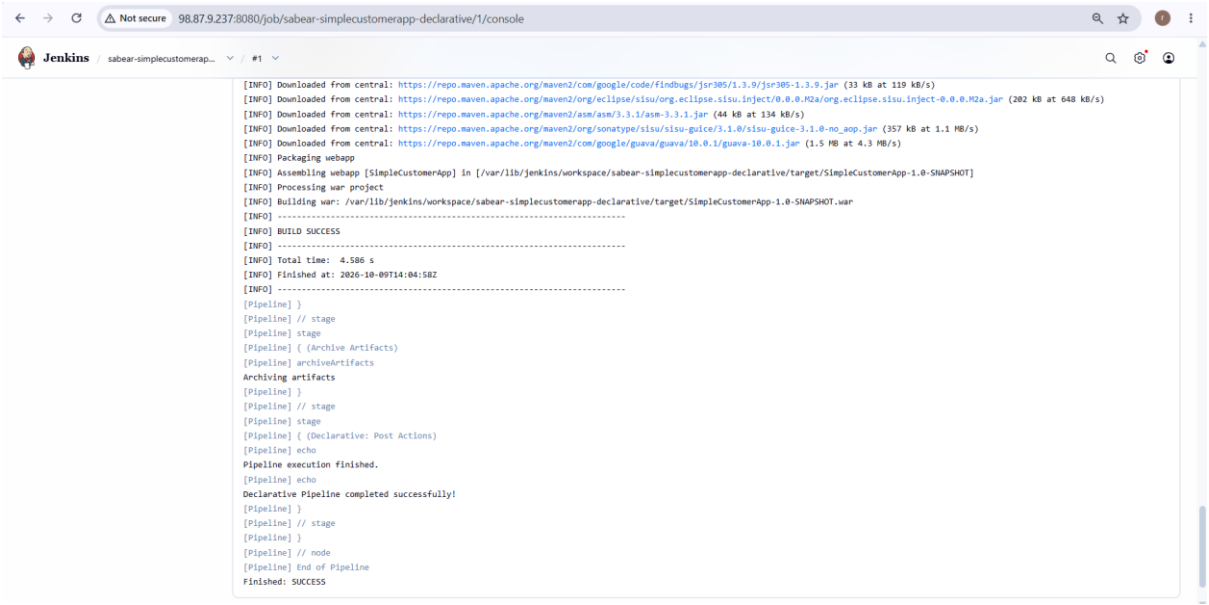
Important: This script assumes a Maven project and a compatible JDK are available on the Jenkins agent. If the repository does not contain pom.xml, first identify its build system and replace the Maven build stage with the appropriate build command.

Step 5: Save and run the job

1. Click Save.
2. On the job dashboard, click Build Now.
3. Under Build History, click the latest build number.

4. Open Console Output.





Step 6: Verify the build

A successful build should show the stages:

- Checkout
- Verify Project
- Build
- Archive Artifacts

The final result should be:

Declarative Pipeline completed successfully!

Pipeline execution finished.

Finished: SUCCESS

These are expected example messages, not a guarantee of success before the build runs.

Step 7: Troubleshooting

Error	Action
Couldn't find any revision to build	Verify that feature-1.1 exists and the repository URL is correct.
mvn: command not found	Install Maven or configure it on the Jenkins agent.
JAVA_HOME is not defined correctly	Correct the JDK configuration for Jenkins.
The goal you specified requires a project to execute	Check whether pom.xml exists in the current workspace or a subdirectory.
BUILD FAILURE	Read the Maven error in Console Output and resolve the reported issue.

6. Create one Declarative job

Source Code: **https://github.com/betawins/Trading-UI.git**

Step 1: Prerequisites

On your Jenkins EC2 instance, check the installed tools:

git --version

node --version

npm --version

If Node.js is not installed, install a version compatible with the project's package.json and lockfile. Ensure the Jenkins agent itself can access these tools.

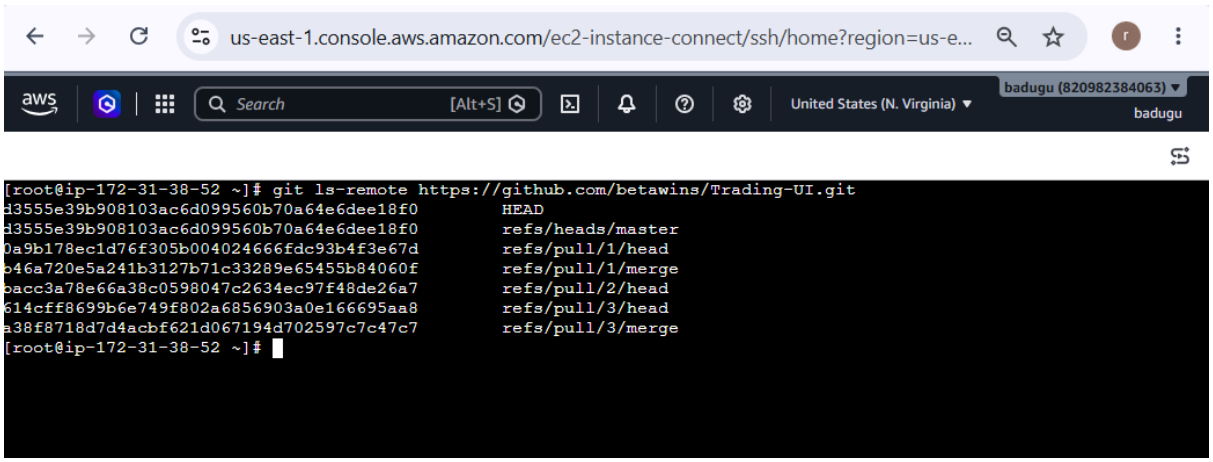
Step 2: Verify the repository

Run this command on the Jenkins machine to confirm that Git can access the repository:

git ls-remote https://github.com/betawins/Trading-UI.git

``

Check the repository on GitHub for its default branch and whether it contains package.json.



Step 3: Create a Jenkins Pipeline job

1. Open the Jenkins dashboard.
2. Click New Item.
3. Enter the job name:

Trading-UI-Declarative-Pipeline

4. Select Pipeline.
5. Click OK.

Step 4: Configure the pipeline script

Scroll down to the Pipeline section and set Definition to Pipeline script.

Paste this Declarative Pipeline script:

```
pipeline {
    agent any
```

```
stages {
    stage('Checkout') {
        steps {
            echo 'Checking out Trading-UI source code'

            git branch: 'main',
                url: 'https://github.com/betawins/Trading-UI.git'
        }
    }

    stage('Verify Project') {
        steps {
            sh '''
                echo "Workspace: $PWD"
                ls -la
                git log -1 --oneline

                if [ ! -f package.json ]; then
                    echo "ERROR: package.json not found at repository root."
                    exit 1
                fi

                node --version
                npm --version
            '''
```

```
    }
}

stage('Install Dependencies') {
    steps {
        sh '''
            if [ -f package-lock.json ]; then
                npm ci
            elif [ -f npm-shrinkwrap.json ]; then
                npm ci
            else
                npm install
            fi
        '''
    }
}

stage('Build UI') {
    steps {
        sh '''
            npm run build
        '''
    }
}

stage('Archive Build') {
```

```
        steps {
            sh '''
                if [ -d dist ]; then
                    echo "Build output found in dist/"
                elif [ -d build ]; then
                    echo "Build output found in build/"
                else
                    echo "ERROR: Expected dist/ or build/ output not found."
                    exit 1
                fi
            '''

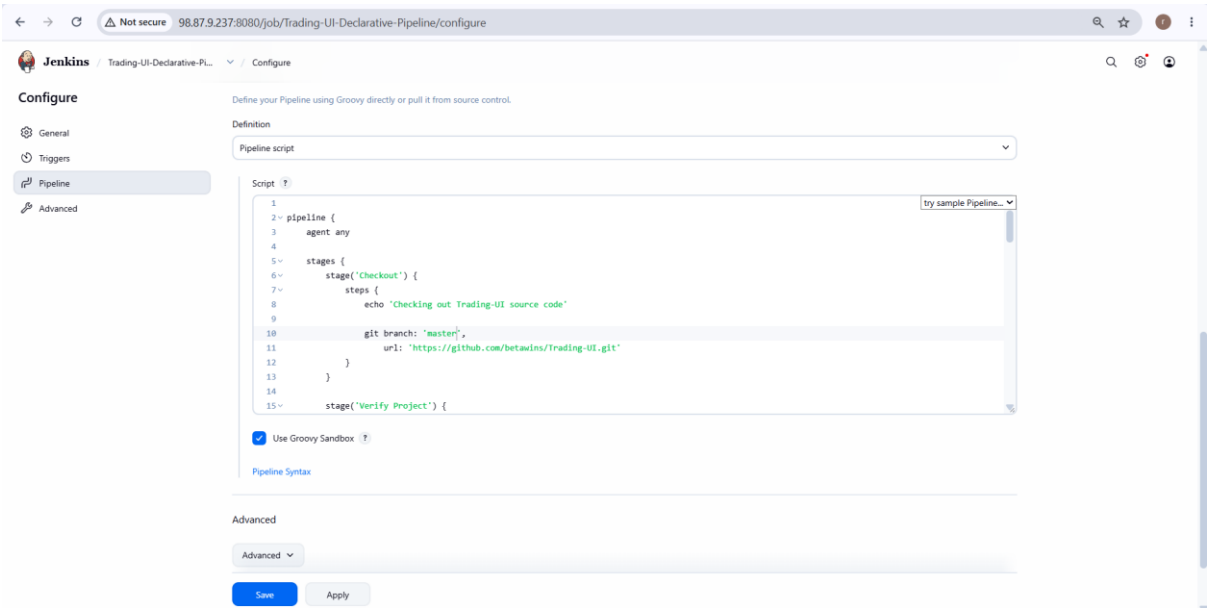
            archiveArtifacts(
                artifacts: 'dist/**,build/**',
                allowEmptyArchive: false
            )
        }
    }
}

post {
    success {
        echo 'Trading-UI pipeline completed successfully!'
    }

    failure {
```

```
        echo 'Trading-UI pipeline failed. Check Console Output.'
    }

    always {
        echo 'Pipeline execution finished.'
    }
  }
}
```



Important before running:

- This script assumes the repository's branch is main. If the default branch is different, replace main with the correct branch name.
- The Jenkins agent needs Node.js and npm installed.
- This example assumes package.json defines a build script and that the build generates a dist/ or build/ directory. Adjust the archive paths if your project uses a different output directory.

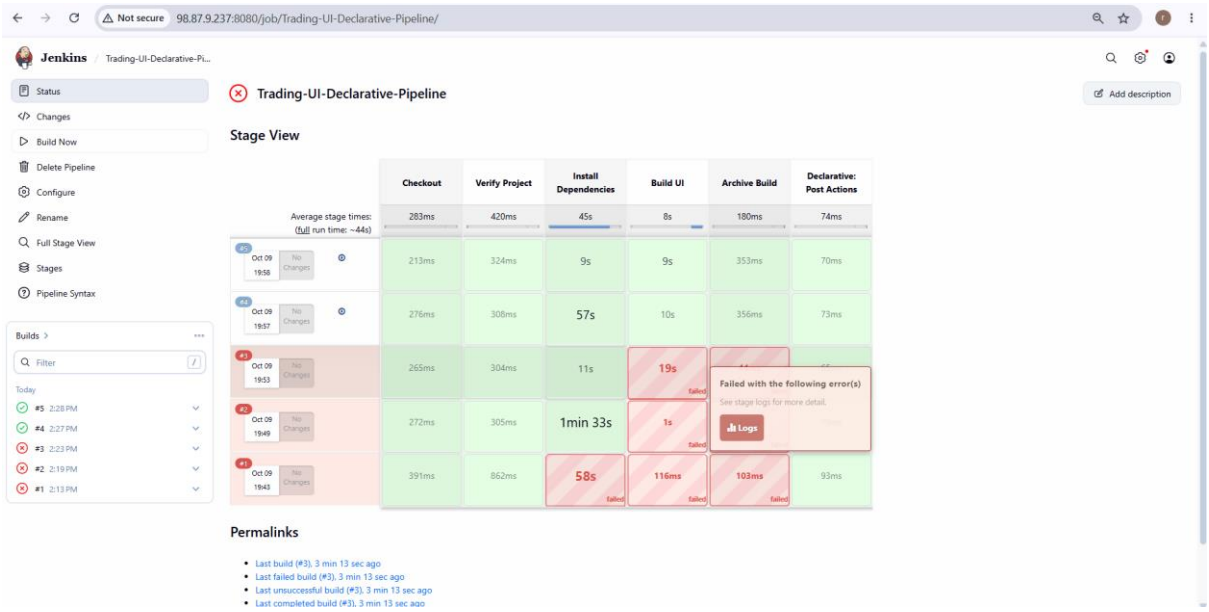
Step 5: Save and run the job

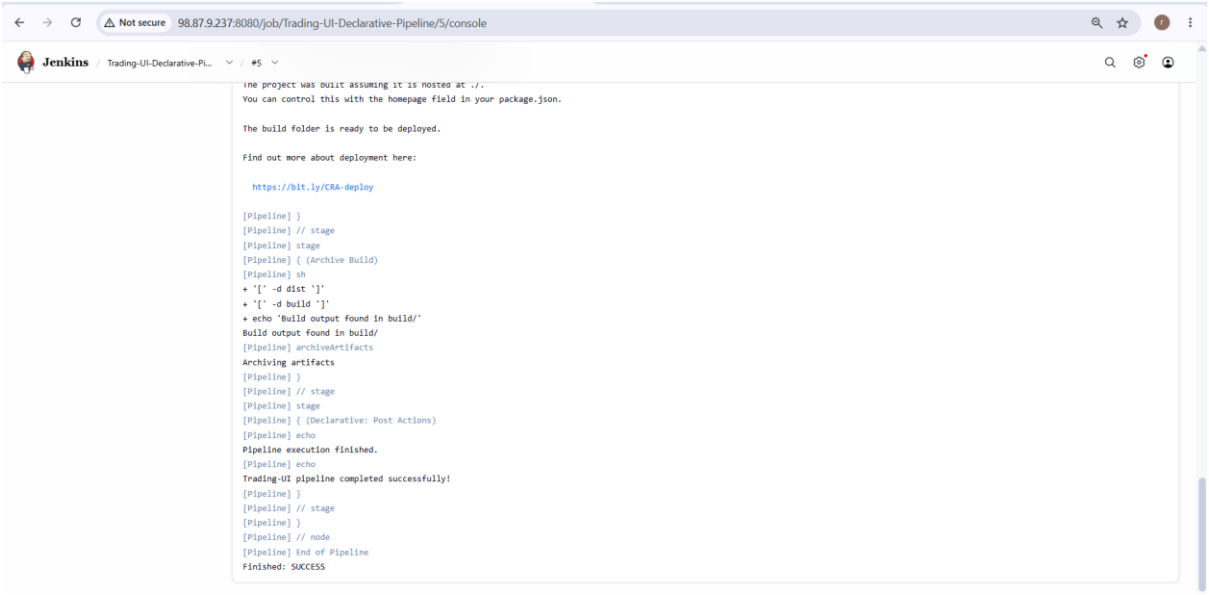
1. Click Save.
2. Open the job dashboard.

3. Click Build Now.
4. Under Build History, select the latest build number.
5. Click Console Output.

Step 6: Verify the stages

You should see these stages in Jenkins:





Step 7: Troubleshooting

Error	Solution

Couldn't find any revision to build	Check the branch name and GitHub URL.
node: command not found	Install Node.js on the Jenkins agent.
npm ci fails	Verify that package-lock.json matches package.json.
Missing script: build	Check the scripts section of package.json and use the correct build command.
dist or build directory not found	Check the framework's configured output directory.

Expected result: One Declarative Jenkins Pipeline job that checks out Trading-UI, installs its dependencies, builds the frontend, and archives the build output.

This job builds the UI; it does not deploy the application to a web server.

6.Execute parallel stages using Jenkins pipeline for any sample job

1. Objective

Create a sample Jenkins job that runs three tasks simultaneously:

- Unit Tests — simulate application unit testing.
- Code Quality Check — simulate a code quality analysis.
- Security Scan — simulate a dependency security scan.

After all three parallel tasks finish, Jenkins will execute a final stage called Package Application.

2. How parallel execution works

Start Pipeline

Checkout / Initialize

Parallel stages run simultaneously

Unit Tests

Code Quality

Security Scan

Package Application

Pipeline Complete

Jenkins waits for all three parallel branches to finish before moving to Package Application.

Prerequisites

Before starting, make sure:

1. Jenkins is running.
2. You can log in to the Jenkins dashboard.
3. A Jenkins agent with a shell is available.
4. You have permission to create a Pipeline job.

This sample uses shell commands only; no Git repository, Maven, Node.js, SonarQube, or security scanner installation is required.

4. Create a Jenkins Pipeline job

1. Open your Jenkins dashboard.
2. Click New Item.
3. Enter the job name: Parallel-Stages-Demo.
4. Select Pipeline.
5. Click OK.
6. Scroll down to the Pipeline section.
7. Set Definition to Pipeline script.
8. Paste the following Jenkinsfile.

5. Jenkinsfile — complete working example

```
pipeline {
    agent any
```

```
options {
    timestamps()
}

stages {
    stage('Initialize') {
        steps {
            echo 'Pipeline started'
            echo "Build Number: ${BUILD_NUMBER}"
            echo "Job Name: ${JOB_NAME}"
        }
    }

    stage('Parallel Checks') {
        parallel {
            stage('Unit Tests') {
                steps {
                    echo 'Unit tests started'
                    sh '''
                        echo "Running unit tests..."
                        sleep 10
                        echo "Unit tests passed"
                    '''
                }
            }
```

```
        stage('Code Quality Check') {
            steps {
                echo 'Code quality check started'
                sh '''
                    echo "Checking code quality..."
                    sleep 15
                    echo "Code quality check passed"
                '''
            }
        }

        stage('Security Scan') {
            steps {
                echo 'Security scan started'
                sh '''
                    echo "Scanning dependencies..."
                    sleep 8
                    echo "Security scan completed"
                '''
            }
        }
    }
}

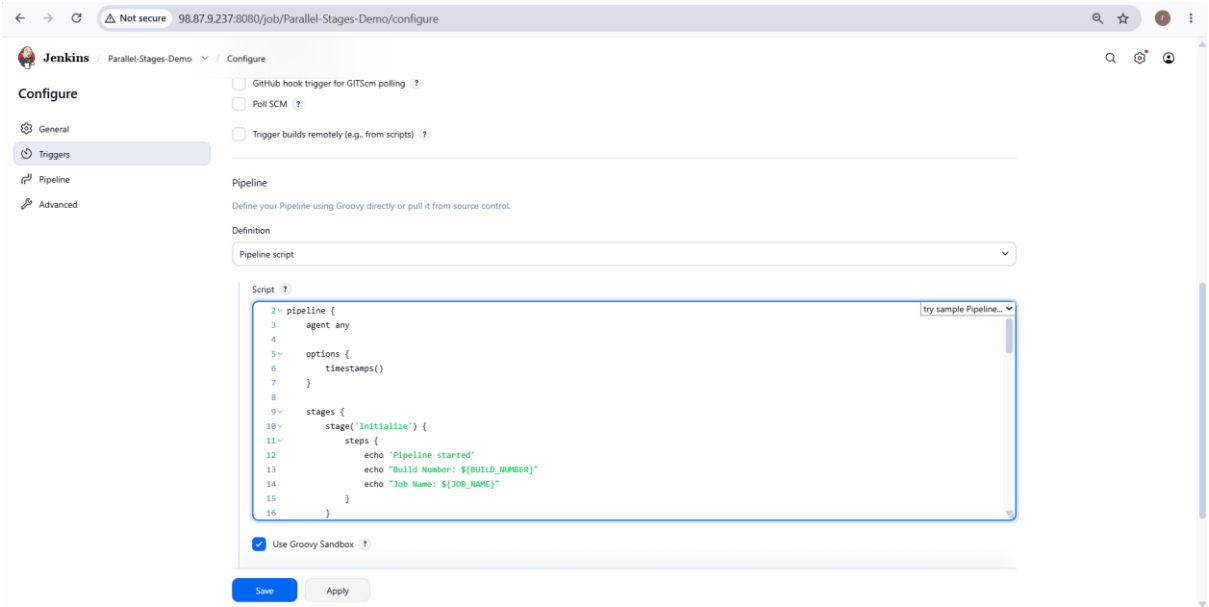
stage('Package Application') {
    steps {
```

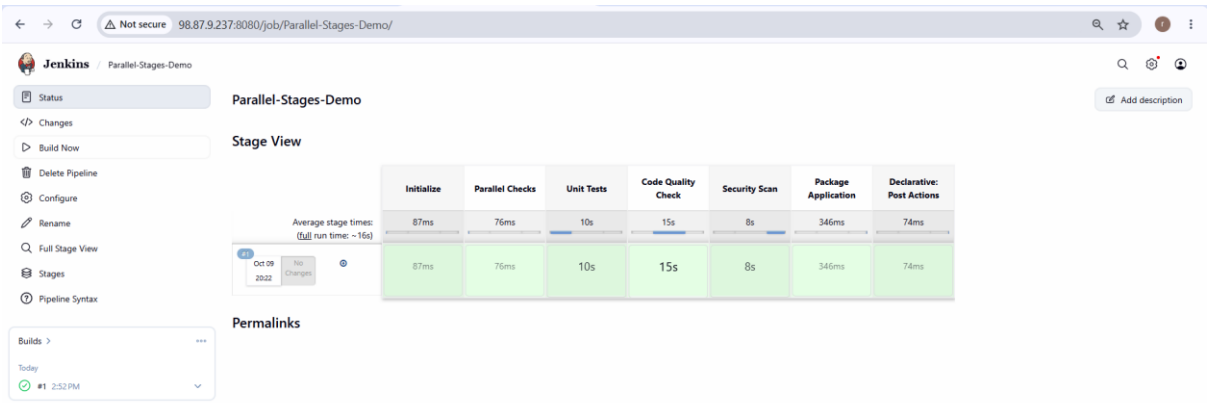
```
            echo 'All parallel stages completed'
            sh '''
                mkdir -p output
                echo "Sample application package" > output/app.txt
                echo "Application packaged successfully"
            '''
            archiveArtifacts artifacts: 'output/**'
        }
    }
}

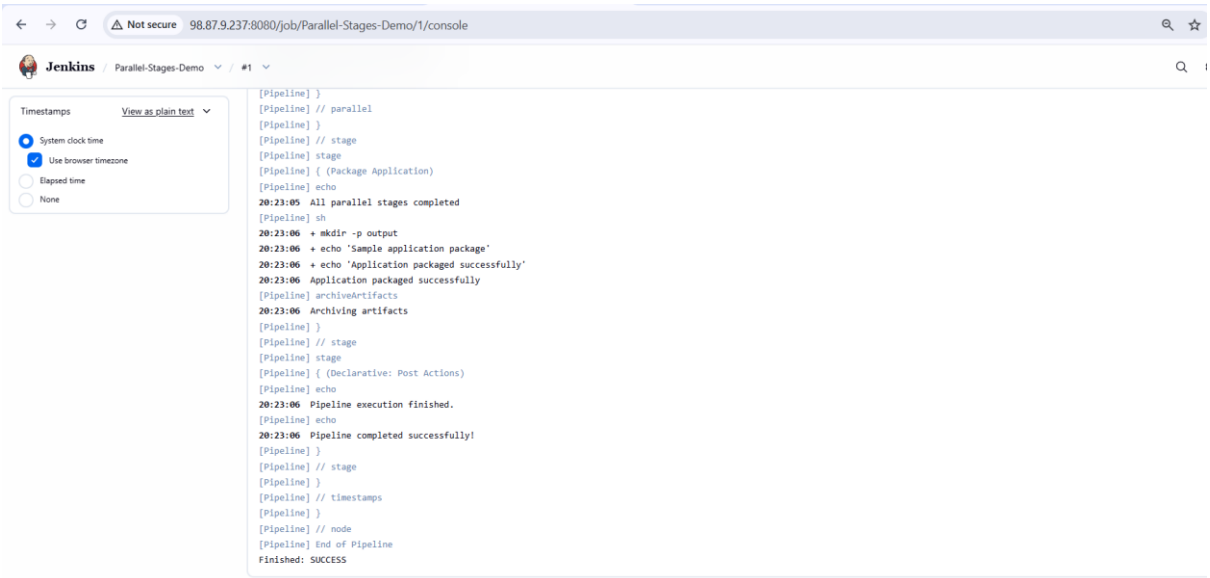
post {
    success {
        echo 'Pipeline completed successfully!'
    }

    failure {
        echo 'Pipeline failed. Check the Console Output.'
    }

    always {
        echo 'Pipeline execution finished.'
    }
}
```


Not secure 98.87.9.237:8080/job/Parallel-Stages-Demo/configure
Jenkins / Parallel-Stages-Demo / Configure
Configure
General
Triggers
Pipeline
Advanced
GitHub hook trigger for GITScm polling
Poll SCM
Trigger builds remotely (e.g., from scripts)
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
options {
timestamps()
}
stages {
stage('Initialize') {
steps {
echo 'Pipeline started'
echo "Build Number: ${BUILD_NUMBER}"
echo "Job Name: ${JOB_NAME}"
}
}
Use Groovy Sandbox
Save
Apply


Not secure 98.87.9.237:8080/job/Parallel-Stages-Demo/
Jenkins / Parallel-Stages-Demo
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Full Stage View
Stages
Pipeline Syntax
Builds
Today
#1 2:52 PM
Parallel-Stages-Demo
Add description
Stage View
Initialize
Parallel Checks
Unit Tests
Code Quality Check
Security Scan
Package Application
Declarative: Post Actions
Average stage times:
(full run time: ~16s)
87ms
76ms
10s
15s
8s
346ms
74ms
#1
Oct 09
2022
No Changes
Permalinks


Not secure 98.87.9.237:8080/job/Parallel-Stages-Demo/1/console
Jenkins / Parallel-Stages-Demo / #1
Timestamps
View as plain text
System clock time
Use browser timezone
Elapsed time
None
[Pipeline] }
[Pipeline] // parallel
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Package Application)
[Pipeline] echo
20:23:05 All parallel stages completed
[Pipeline] sh
20:23:06 + mkdir -p output
20:23:06 + echo 'Sample application package'
20:23:06 + echo 'Application packaged successfully'
20:23:06 Application packaged successfully
[Pipeline] archiveArtifacts
20:23:06 Archiving artifacts
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
20:23:06 Pipeline execution finished.
[Pipeline] echo
20:23:06 Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

Note: The sleep commands simulate work for demonstration purposes. The unit tests, quality check, and security scan are not real tests or scans.

8. Verify the archived artifact

After the successful build:

1. Open the build page.
2. Look for Build Artifacts.
3. Click output/app.txt.
4. Confirm that it contains:

Sample application package

9. Important interview points

Question	Answer
Why use parallel stages?	To reduce total pipeline execution time by running independent tasks concurrently.
How do you define parallel stages?	Use the parallel { ... } directive inside a Declarative Pipeline stage.
Can stages run in parallel on the same agent?	Yes, depending on executor availability and the pipeline configuration.
What if one parallel branch fails?	By default, the parallel stage fails and the pipeline generally stops progressing to subsequent stages.
What must be true for actual concurrency?	Jenkins needs sufficient available executors and agents to run the branches simultaneously.

Production example: In a real CI pipeline, you could run unit tests, SonarQube analysis, and a dependency vulnerability scan in parallel, then build and publish the application only after the required checks pass.

7.Execute Jenkins pipeline stages using shared libraries

- **Reference:** https://phoenixnap.com/kb/jenkins-shared-library

The objective is to create a Jenkins Shared Library and reuse its functions inside a Declarative Pipeline. This helps you avoid writing the same pipeline logic in multiple Jenkins jobs.

For example, instead of writing checkout, testing, and build steps separately in every Jenkinsfile, you can define reusable functions in a shared Git repository and call them from different jobs.

1. Understand the architecture

GitHub Shared Library Repository

vars/checkoutCode.groovy

vars/runTests.groovy

vars/buildApp.groovy

Jenkins Global Pipeline Libraries

Configure library name, Git URL and branch

Jenkinsfile

@Library('devops-shared-library') _

Calls reusable library functions

Pipeline Executes

Checkout → Test → Build → Archive

2. Prerequisites

- A running Jenkins server with permission to configure system settings and create jobs.
- Git installed on the Jenkins controller or agent as required by your setup.
- A GitHub repository for the shared library.
- A separate Jenkins Pipeline job that will use the library.

We will use a sample project so you can learn the shared-library mechanism without needing to modify your Trading-UI application.

3. Create the Shared Library repository in GitHub

Create a new repository named jenkins-shared-library.

1. Open GitHub.
2. Click New repository.
3. Repository name: jenkins-shared-library.
4. For this lab, select Public so Jenkins can access it without GitHub credentials. Use a private repository with configured credentials for production.
5. Click Create repository.

The repository will contain this structure:

```
jenkins-shared-library/
└── vars/
    ├── prepareBuild.groovy
    ├── runUnitTests.groovy
    ├── codeQualityCheck.groovy
    ├── securityCheck.groovy
    └── packageApp.groovy
```

Jenkins exposes scripts under vars/ as reusable pipeline steps. Each file's name becomes the function name you call from the Jenkinsfile.

4. Create the shared library functions

In your GitHub repository, create the vars directory and add the following files. You can create each file by clicking Add file → Create new file and entering its path.

File 1: vars/prepareBuild.groovy

```
def call() {
    echo 'Preparing the application build'
    sh '''
        mkdir -p output
        echo "Build workspace initialized"
    '''
```

```
}
```

File 2: vars/runUnitTests.groovy

```
def call() {
    echo 'Running unit tests'
    sh '''
        echo "Simulating unit tests..."
        sleep 5
        echo "Unit test stage completed"
    '''
}
```

File 3: vars/codeQualityCheck.groovy

```
def call() {
    echo 'Running code quality check'
    sh '''
        echo "Simulating code quality analysis..."
        sleep 5
        echo "Code quality stage completed"
    '''
}
```

File 4: vars/securityCheck.groovy

```
def call() {
    echo 'Running security check'
    sh '''
        echo "Simulating dependency security scan..."
        sleep 5
        echo "Security check stage completed"
```

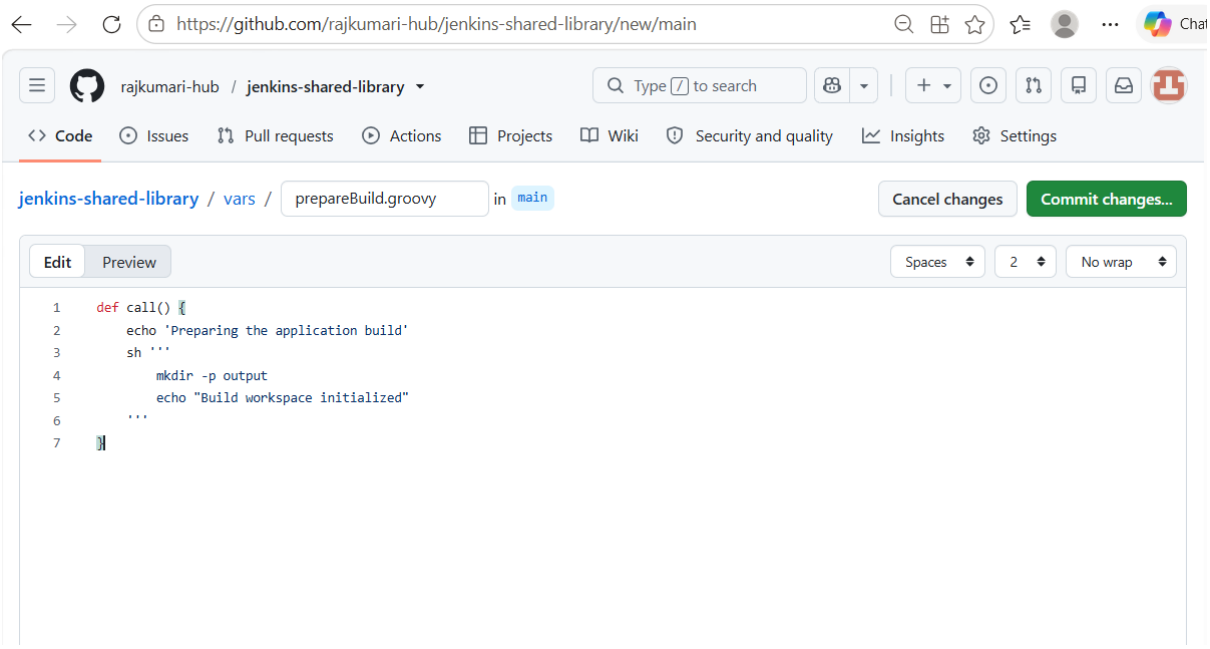
```
    '''
}
```

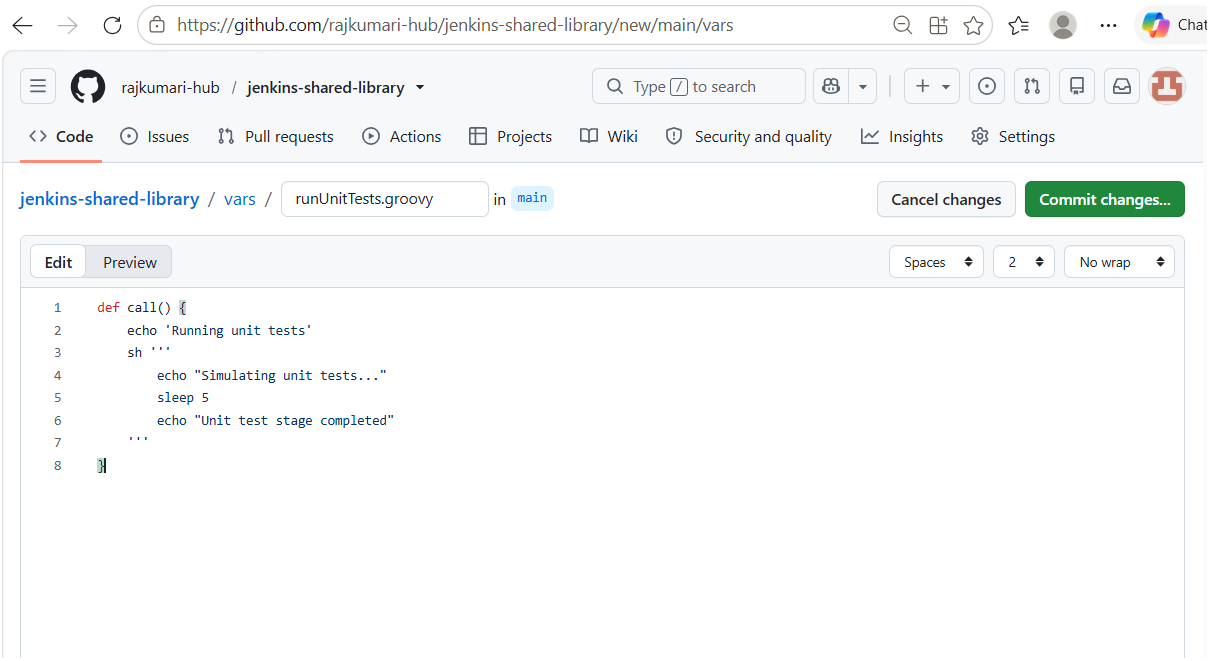
File 5: vars/packageApp.groovy

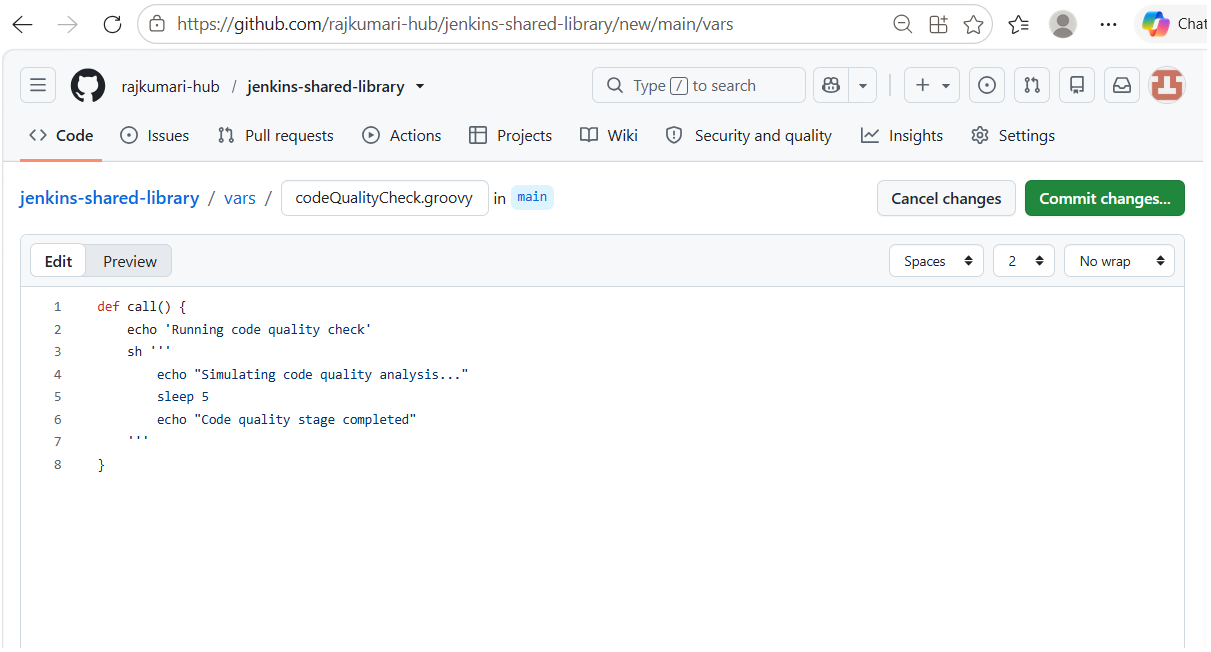
```
def call() {
    echo 'Packaging application'
    sh '''
        mkdir -p output
        echo "Sample application package" > output/app.txt
        echo "Application package created"
    '''

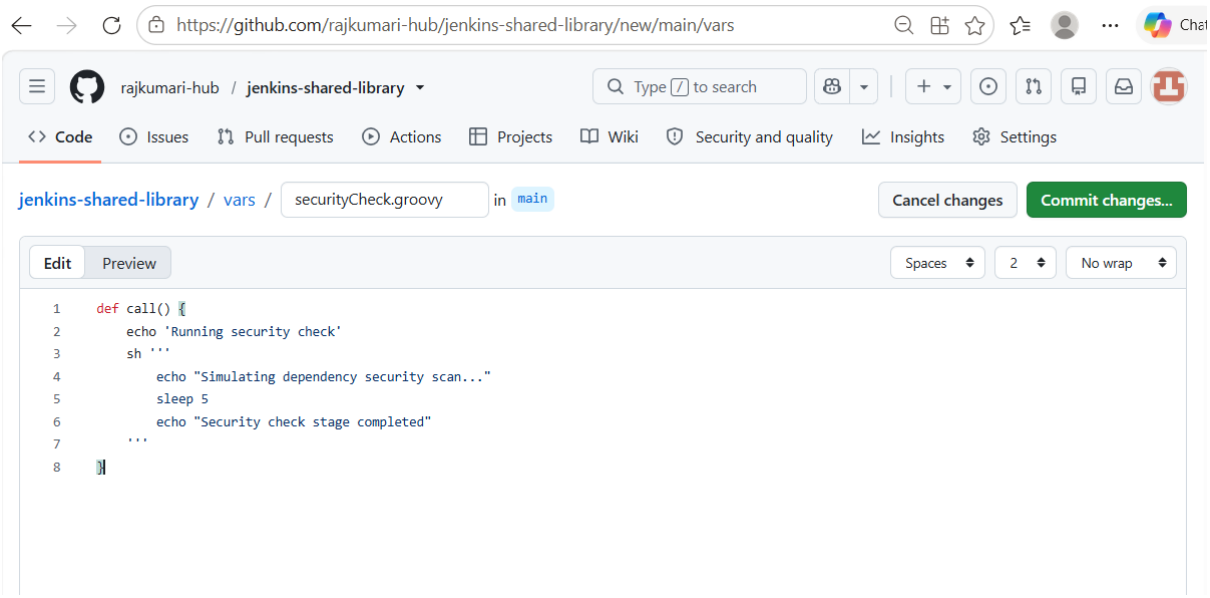
    archiveArtifacts artifacts: 'output/**'
}
```

Commit each file to your repository. These are demonstration functions; the checks simulate work rather than testing real application code or scanning actual dependencies.




https://github.com/rajkumari-hub/jenkins-shared-library/new/main/vars
rajkumari-hub / jenkins-shared-library
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
jenkins-shared-library / vars / runUnitTests.groovy in main
Cancel changes
Commit changes...
Edit
Preview
Spaces
2
No wrap
def call() {
echo 'Running unit tests'
sh '''
echo "Simulating unit tests..."
sleep 5
echo "Unit test stage completed"
'''
}


https://github.com/rajkumari-hub/jenkins-shared-library/new/main/vars
rajkumari-hub / jenkins-shared-library
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
jenkins-shared-library / vars / codeQualityCheck.groovy in main
Cancel changes
Commit changes...
Edit
Preview
Spaces
2
No wrap
def call() {
echo 'Running code quality check'
sh '''
echo "Simulating code quality analysis..."
sleep 5
echo "Code quality stage completed"
'''
}



5. Configure the Shared Library in Jenkins

Now tell Jenkins where to find your GitHub library. The global library configuration is documented in the Jenkins Shared Libraries guide.

1. Open the Jenkins dashboard.
2. Click Manage Jenkins.
3. Select System or Configure System, depending on your Jenkins version.
4. Find Global Trusted Pipeline Libraries or Global Pipeline Libraries.
5. Click Add and enter the following values.

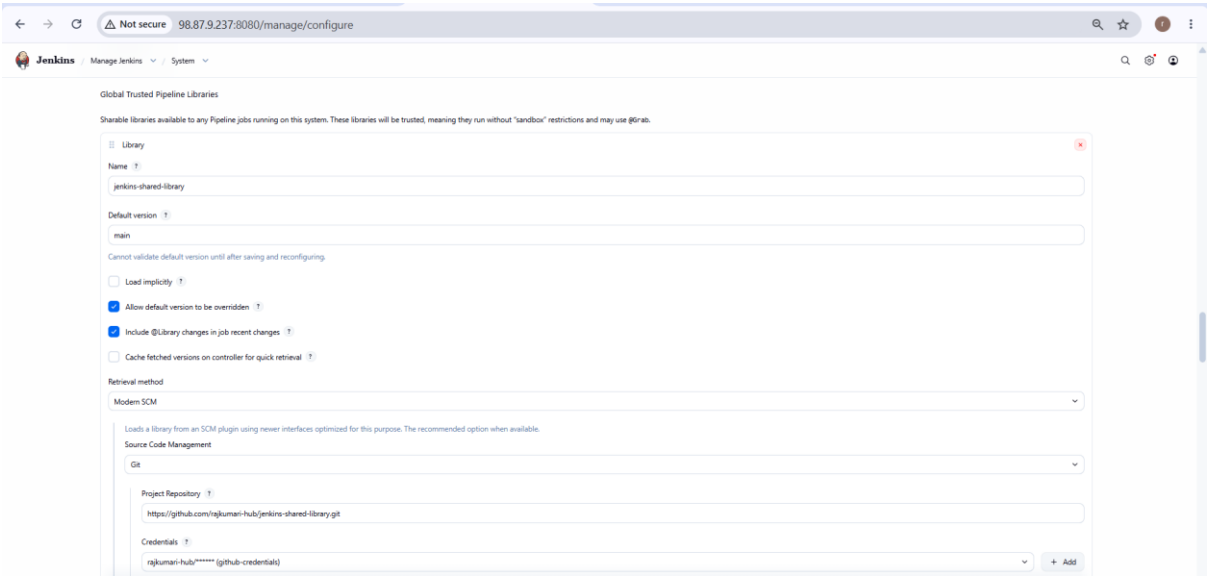
Setting	Value
Name	devops-shared-library
Default version	main
Load implicitly	Unchecked
Allow default version to be overridden	Checked
Retrieval method	Modern SCM
Source Code Management	Git

Project Repository	https://github.com/YOUR-GITHUB-USERNAME/jenkins-shared-library.git
Credentials	None for a public repository

Replace YOUR-GITHUB-USERNAME with your actual GitHub username. If your repository uses master instead of main, use master as the default version.

Click Save.

Security note: Trusted libraries can access powerful Jenkins APIs. Only configure a library as trusted if you control and trust its maintainers and repository.



6. Create a Jenkins Pipeline job

1. Go to the Jenkins dashboard.
2. Click New Item.
3. Enter the job name Shared-Library-Pipeline-Demo.
4. Select Pipeline, then click OK.
5. Scroll to the Pipeline section.
6. Set Definition to Pipeline script.

7. Complete Jenkinsfile

```
@Library('devops-shared-library') _
```

```
pipeline {
    agent any

    stages {
        stage('Initialize') {
            steps {
                echo 'Starting Shared Library Pipeline'
                prepareBuild()
            }
        }

        stage('Unit Tests') {
            steps {
                runUnitTests()
            }
        }

        stage('Code Quality') {
            steps {
                codeQualityCheck()
            }
        }

        stage('Security Scan') {
            steps {
```

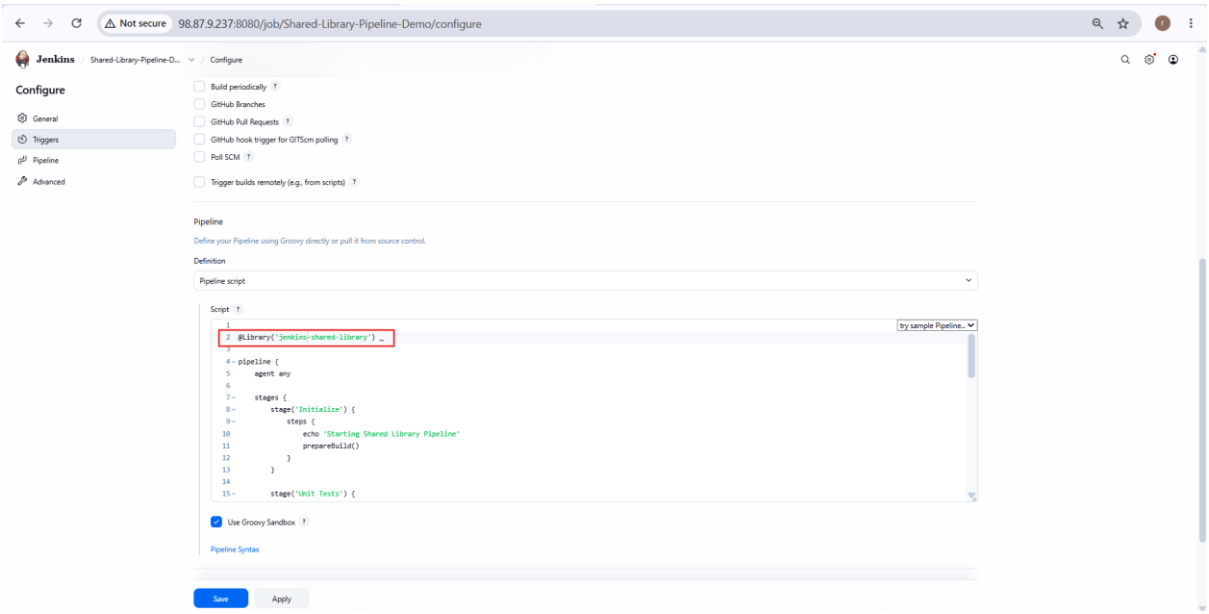
```
                securityCheck()
            }
        }

        stage('Package Application') {
            steps {
                packageApp()
            }
        }
    }

    post {
        success {
            echo 'Shared Library Pipeline completed successfully!'
        }

        failure {
            echo 'Pipeline failed. Check Console Output.'
        }

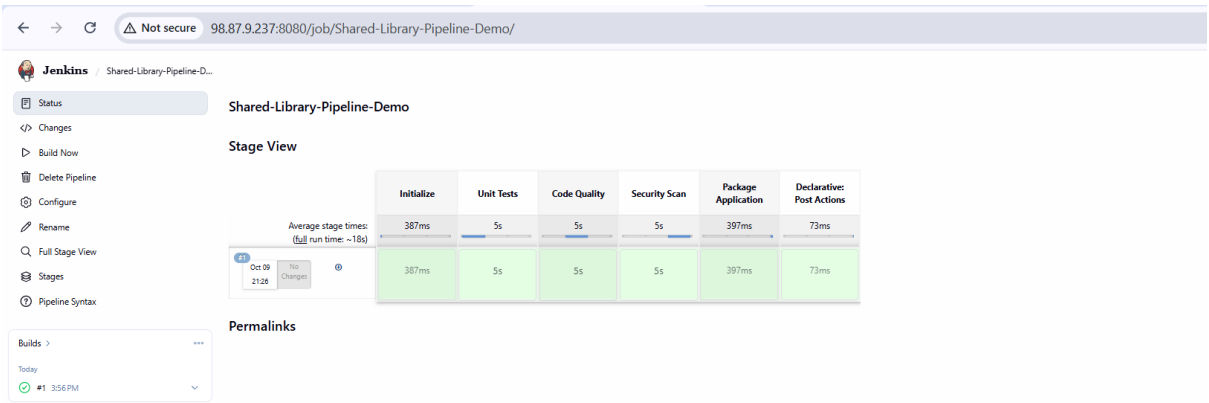
        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

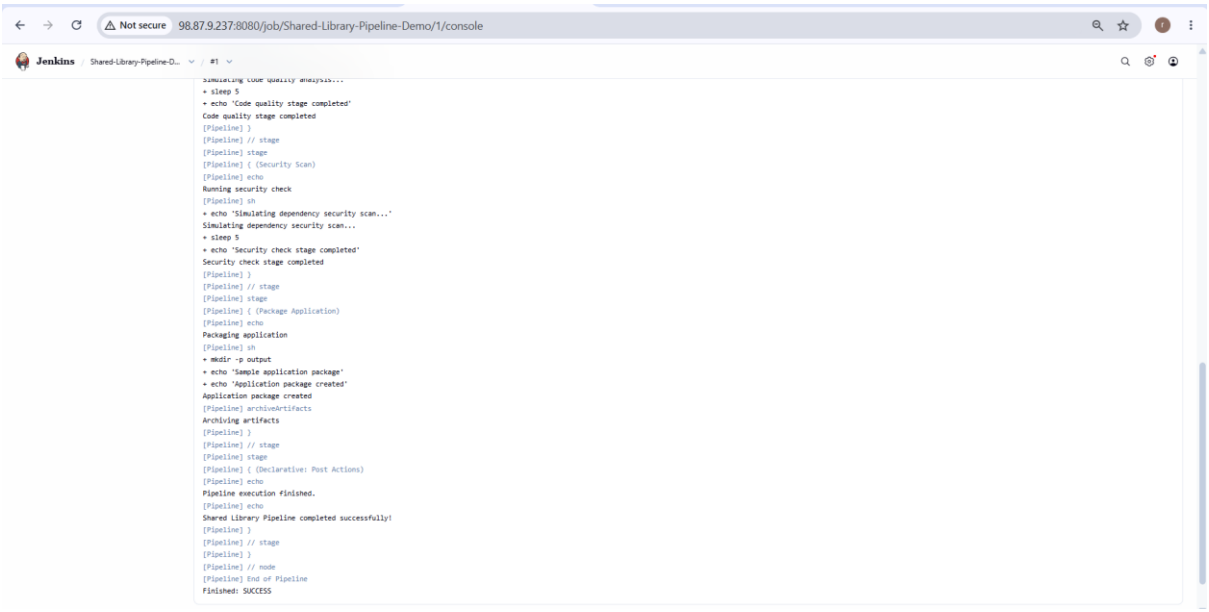


The first line loads the library by its configured name. Calls such as runUnitTests() and packageApp() execute the functions from the corresponding Groovy files in vars/

8. Run the pipeline

1. Click Save.
2. Click Build Now.
3. Open the latest build.
4. Click Console Output.





You should see the stages execute in order:

Starting Shared Library Pipeline

Preparing the application build

Running unit tests

Unit test stage completed

Running code quality check

Code quality stage completed

Running security check

Security check stage completed

Packaging application

Application package created

Shared Library Pipeline completed successfully!

Finished: SUCCESS

The actual output includes Jenkins pipeline log markers and timestamps.

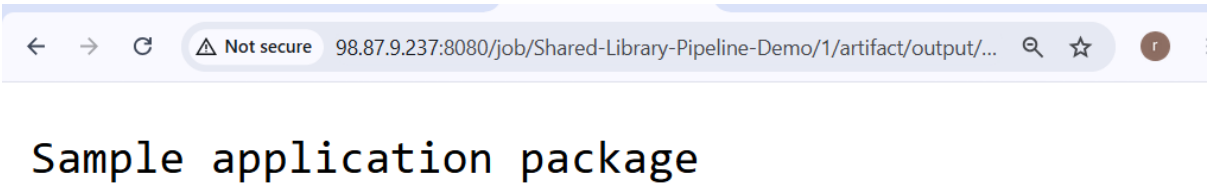
9. Verify the build artifact

After the job succeeds:

1. Open the build page.

2. Look for Build Artifacts.
3. Click output/app.txt.
4. Confirm the content is:

Sample application package



10. Troubleshooting

Error	What to check
Library ... not found	Confirm the library name in Jenkins matches @Library('devops-shared-library').
Git checkout or authentication error	Verify the repository URL, branch, and credentials if private.
No such DSL method for a library function	Confirm the matching .groovy file exists under vars/, the spelling matches, and the file contains a call() method.
No such file or directory	Confirm the vars/ directory is at the configured library root and all files have been committed.
archiveArtifacts finds no files	Verify packageApp.groovy creates output/app.txt before archiving.